



UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

DIPARTIMENTO DI SCIENZE TEORICHE E APPLICATE

CORSO DI STUDIO TRIENNALE IN
INFORMATICA

**Progettazione e implementazione di una
piattaforma web per la divulgazione
medico-scientifica**

Relatore:

Prof.ssa Alessandra Rizzardi

Correlatore:

Prof.ssa Sabrina Sicari

Tesi di Laurea di:

Rahul Kabotra
Matricola 757605

Anno accademico:

2025/2026

“The Web does not just connect machines, it connects people.”
- Tim Berners-Lee (*Inventore del World Wide Web, Knight Foundation, 2008*)

Indice

1	Introduzione	1
2	Stato dell'arte	4
2.1	Piattaforme generaliste	4
2.2	Piattaforme specialistiche a pagamento	5
2.3	Piattaforme accademiche e istituzionali	5
2.4	Confronto tra le piattaforme	6
3	Tecnologie adottate	8
3.1	Tecnologie per l'autenticazione web	8
3.1.1	Autenticazione basata su sessione	8
3.1.2	Autenticazione basata su token: JSON Web Token	9
3.1.3	Firma del token: HMAC-SHA256	9
3.1.4	Hashing sicuro delle password	10
3.1.5	Scelta adottata in NETMED	10
3.2	Persistenza dei dati	12
3.2.1	Basi di dati relazionali	12
3.2.2	Basi di dati NoSQL	12
3.2.3	PostgreSQL e il tipo di dato JSONB	12
3.2.4	Applicazione a NETMED	13
3.3	Architetture front-end	14
3.3.1	Multi-Page Application	14
3.3.2	Single Page Application	14
3.3.3	JavaScript vanilla e scelta adottata in NETMED	15
3.4	Containerizzazione	16
3.4.1	Il modello Docker	16
3.4.2	Docker Compose	16
3.4.3	Ambiente di sviluppo	17
3.5	Hosting dei contenuti video	17

4	Progettazione e implementazione	18
4.1	Architettura generale del sistema	18
4.1.1	Componenti principali	18
4.1.2	Flusso di una richiesta	20
4.1.3	Ambiente di esecuzione	20
4.1.4	Pool di connessioni	21
4.2	Modello dei dati	21
4.2.1	Utenti e verifica professionale	22
4.2.2	Contenuti e catalogazione	23
4.2.3	Interazione degli utenti	23
4.2.4	Moderazione e amministrazione	24
4.2.5	Sistema delle notifiche	24
4.3	Modellazione UML delle entità principali	24
4.3.1	Classi del dominio	25
4.3.2	Relazioni tra le entità	26
4.4	Sequence diagram dei flussi chiave	26
4.4.1	Flusso di autenticazione dell'utente	27
4.4.2	Richiesta di verifica del profilo creator	27
4.4.3	Segnalazione di un contenuto	29
4.4.4	Moderazione dei commenti e strike	32
4.5	Struttura del backend e progettazione dell'API REST	32
4.5.1	Organizzazione dei moduli	33
4.5.2	Middleware	34
4.5.3	Rotte suddivise per ruolo	34
4.5.4	Servizi condivisi e accesso ai dati	36
4.5.5	Limitazione del rate di richieste	36
4.6	Struttura del frontend	36
4.6.1	Organizzazione delle pagine e delle risorse	36
4.6.2	Interfaccia utente	37
4.6.3	Pannello di amministrazione	39
4.6.4	Componenti trasversali	41
4.7	Sistema di verifica dei creator	41
4.7.1	Modello dei dati	41
4.7.2	Fase di richiesta	42
4.7.3	Fase di valutazione	42
4.7.4	Approvazione e revoca della verifica	42
4.8	Sistema di notifiche in app	43
4.8.1	Modello dei dati e tipi di notifica	43
4.8.2	Creazione secondo il paradigma best-effort	44

4.8.3	Consumo dal client	44
4.9	Implementazione dei componenti più significativi	44
4.9.1	Middleware di autenticazione	44
4.9.2	Creazione delle notifiche	45
4.9.3	Caricamento di un video	46
4.9.4	Registrazione utente con validazione e hashing bcrypt	49
5	Valutazione del sistema	51
5.1	Obiettivi della valutazione	51
5.2	Ambiente e metodologia di test	51
5.2.1	Copertura delle funzionalità principali	52
5.2.2	Verifica delle politiche di autorizzazione	52
5.2.3	Esito della suite	53
5.3	Tempi di risposta degli endpoint principali	53
5.3.1	Metodologia di misurazione	53
5.3.2	Risultati	53
5.3.3	Discussione dei risultati	54
5.4	Ottimizzazione dell'accesso ai dati	55
5.4.1	Indici sulle colonne di ricerca	55
5.4.2	Effetto dell'indice sul feed pubblico	55
5.4.3	Pool di connessioni	57
5.5	Sintesi delle valutazioni	57
6	Conclusioni e sviluppi futuri	59
6.1	Risultati raggiunti	59
6.2	Limiti dell'attuale implementazione	60
6.2.1	Limiti funzionali	60
6.2.2	Limiti tecnici	61
6.3	Sviluppi futuri	61
6.4	Considerazioni finali	62

Elenco delle figure

4.1	Architettura a tre livelli di NETMED. Il client comunica con il server tramite il protocollo HTTP; il server accede al database tramite una connessione TCP. Tutto avviene all'interno di una rete Docker.	19
4.2	Schema entità-relazione delle entità principali. Le cardinalità (<i>min</i> , <i>max</i>) sono riportate sull'arco in corrispondenza dell'entità a cui si riferiscono. .	22
4.3	Diagramma delle classi del dominio applicativo.	25
4.4	Flusso di autenticazione dell'utente.	28
4.5	Richiesta e approvazione della verifica del profilo creator.	30
4.6	Segnalazione di un contenuto e valutazione da parte dell'amministratore. .	31
4.7	Home page di NETMED: in alto la hero image, in basso la sezione con i contenuti in evidenza.	38
4.8	Pagina di registrazione, con il modulo aggiuntivo per la richiesta di verifica.	38
4.9	Pagina di dettaglio di un video: in alto il player, il titolo, il riferimento al creator con il tesserino di verifica e i pulsanti di interazione; in basso la descrizione, i tag e la sezione dei commenti.	39
4.10	Ricerca: i risultati compaiono in un riquadro a comparsa, suddivisi per video e per tag.	40
4.11	Pagina di caricamento di un nuovo video.	40
4.12	Pannello di amministrazione: la sidebar seleziona la sezione, il contenuto centrale viene aggiornato senza ricaricare la pagina.	41
5.1	Tempo medio di risposta degli endpoint principali, ordinati per valore crescente. L'endpoint di autenticazione è di un ordine di grandezza più lento degli altri.	54
5.2	Costo dell'interrogazione del feed pubblico nelle quattro configurazioni misurate. L'indice parziale riporta il tempo sotto il quarto di millisecondo.	56

Elenco delle tabelle

2.1	Confronto delle piattaforme più rappresentative di ciascuna categoria, rispetto ai quattro requisiti individuati.	6
4.1	Codici di stato HTTP utilizzati nella piattaforma.	33
4.2	Endpoint REST principali del backend.	35
4.3	Tipologie di notifiche presenti in NETMED.	43
5.1	Ripartizione dei casi di test della suite automatica.	53
5.2	Tempi di risposta degli endpoint principali, misurati con dieci connessioni concorrenti per dieci secondi su un catalogo di 50.002 video.	54
5.3	Costo dell'interrogazione del feed pubblico nelle quattro configurazioni misurate.	56

Elenco dei codici

4.1	Middleware di autenticazione JWT (<code>middleware/authMiddleware.js</code>). . .	44
4.2	Helper per la creazione delle notifiche (<code>db/notifications.js</code>).	45
4.3	Caricamento di un video (<code>routes/creatorRoutes.js</code>).	46
4.4	Registrazione utente con validazione e hashing (<code>routes/auth.js</code>).	49
5.1	Interrogazione del feed pubblico (<code>routes/userRoutes.js</code>).	55
5.2	Indice parziale aggiunto allo schema (<code>db-init/01_schema.sql</code>).	57

1

Introduzione

Negli ultimi vent'anni la diffusione di internet e dei dispositivi mobili ha cambiato il modo in cui la popolazione accede alle informazioni in ambito sanitario. Secondo i dati Eurostat [1], il 55% dei cittadini dell'Unione Europea dichiara di utilizzare il web per informarsi sui temi legati alla salute. Diversi studi [2, 3] indicano nelle piattaforme di condivisione video, e in particolare in YouTube, uno dei canali più frequentati per approfondire la sintomatologia, le opzioni terapeutiche e i corretti stili di vita. Questi lavori evidenziano come i video divulgativi siano diventati lo strumento principale di comunicazione sanitaria di massa, spesso preferiti alla classica lettura di riviste specialistiche per la loro immediatezza di accesso.

Questa trasformazione presenta però un aspetto critico: attualmente le piattaforme non prevedono in modo sistematico la verifica della qualifica professionale di chi produce e diffonde contenuti a tema medico. Gli unici sistemi di verifica dell'identità oggi presenti nei principali social network sono le spunte blu, che confermano solo l'autenticità del profilo, senza nessuna garanzia riguardo alla certificazione delle competenze professionali dell'autore o all'iscrizione ai relativi albi professionali. Nello stesso feed, video prodotti da medici, ricercatori e istituzioni sanitarie coesistono con contenuti realizzati da autori privi di competenza in un determinato ambito. Questa coesistenza indistinta genera nell'utente finale una condizione di disorientamento informativo, che numerose ricerche hanno collegato alla diffusione di disinformazione sanitaria [4]. Il termine "infodemia", coniato nel 2003 da Rothkopf [5] per descrivere la diffusione non controllata di informazioni non corrette, è stato ripreso dall'OMS durante la pandemia di SARS-CoV-2 [6],

che ne ha fatto una nozione di uso corrente.

Il quadro descritto evidenzia una problematica: l'assenza di piattaforme di streaming video, specializzate nell'ambito medico-scientifico, che integrino meccanismi di verifica della qualifica professionale del creator e la mancanza di strumenti per la moderazione dei contenuti. Le fonti informative disponibili, in questo momento, sono: le piattaforme generaliste, prive di controllo di dominio (YouTube), e le piattaforme accademiche a pagamento, ad esempio Streamed, orientate esclusivamente ad un pubblico specialistico di professionisti sanitari, e non destinate alla divulgazione al grande pubblico. Serve una via intermedia che permetta sia l'utilizzo da parte di qualsiasi utente, sia l'affidabilità delle fonti, verificate da creator che possiedono una qualifica professionale certificata.

Il presente lavoro di tesi propone la progettazione e l'implementazione di NETMED, una piattaforma web di streaming di contenuti divulgativi a carattere medico-scientifico, che è stata sviluppata come modello alternativo alle piattaforme generaliste. La piattaforma introduce diversi aspetti: un sistema di verifica degli utenti effettuato dall'admin che garantisce la moderazione dei contenuti e dei commenti postati nella piattaforma, e una separazione strutturale tra utente pubblico, creator verificato e amministratore. NETMED si propone come modello alternativo alle piattaforme esistenti: il suo scopo è portare la validazione della qualifica professionale del creator al centro del patto informativo con l'utente, in un servizio rivolto a un pubblico ampio. Il sistema di verifica del creator è l'elemento centrale dell'intera piattaforma. Il suo scopo è duplice: rendere identificabile e responsabile chi pubblica, così che l'utente sappia quale qualifica professionale c'è dietro a un contenuto, e mantenere al tempo stesso la semplicità d'uso. La verifica non entra nel merito della correttezza dei singoli video: si tratta di un limite noto, discusso nel Capitolo 6 insieme alle possibili evoluzioni.

Per la realizzazione del sistema si è scelto di adottare uno stack basato su strumenti open source. Il backend è stato sviluppato in Node.js con il framework Express [7]. La scelta nasce dalla natura non bloccante di Node.js, che lo rende adatto a servizi di rete ad elevata concorrenza. La persistenza dei dati viene affidata a PostgreSQL, che è un sistema di gestione di database relazionale scelto per le sue funzionalità transazionali e per il supporto nativo a tipi di dato complessi come JSONB. Il frontend è stato realizzato in HTML, CSS e JavaScript vanilla, senza l'utilizzo di framework client-side, così da mantenere una corretta leggibilità del codice e ridurre le dipendenze esterne. Per quanto riguarda l'autenticazione si usa lo standard JSON Web Token con algoritmo di firma HS256, mentre le password sono conservate tramite hashing con la funzione bcrypt a dieci round. L'intero sistema viene gestito all'interno di un ambiente Docker Compose, per consentire la riproducibilità dell'ambiente e prevenire le varie problematiche di configurazione sulle diverse macchine in cui l'applicazione viene eseguita. Le tecnologie adottate e la motivazione della loro scelta sono descritte nel Capitolo 3.

Il presente lavoro è organizzato come segue:

- il Capitolo 2 presenta lo stato dell'arte relativo alle piattaforme di streaming video in ambito medico, corredato da un confronto sintetico tra le soluzioni esistenti e le loro limitazioni;
- il Capitolo 3 descrive le tecnologie adottate per la realizzazione di NETMED e le motivazioni che ne hanno guidato la scelta;
- il Capitolo 4 descrive l'architettura del sistema, il modello dei dati e i principali flussi di interazione, corredati da diagrammi UML, sequence diagram, mockup grafici dell'interfaccia e parti significative del codice sorgente;
- il Capitolo 5 riporta la valutazione delle performance del sistema, con riferimento ai test funzionali automatizzati e alle misurazioni dei tempi di risposta degli endpoint principali;
- il Capitolo 6 presenta le conclusioni del lavoro, le limitazioni riscontrate e i possibili sviluppi futuri.

2

Stato dell'arte

Per comprendere nel modo migliore possibile il progetto, è stata effettuata un'analisi delle principali piattaforme di streaming video per la diffusione di contenuti medico-scientifici attualmente disponibili sul mercato. Le soluzioni possono essere raggruppate in tre principali macrocategorie in base al modello di business e al pubblico di riferimento: le piattaforme generaliste, i siti specialistici a pagamento e i canali accademici o istituzionali. Il confronto di queste categorie è riportato nella Tabella 2.1 al termine del capitolo.

2.1 Piattaforme generaliste

Le piattaforme generaliste comprendono servizi di condivisione video aperti al pubblico e offrono un'ampia varietà di contenuti. Un esempio è YouTube, che risulta la seconda piattaforma più visitata al mondo [8] e dichiara miliardi di utenti autenticati ogni mese [9]. Numerosi studi [3] hanno documentato l'utilizzo di YouTube come fonte di informazione sanitaria da parte del pubblico. Il modello di YouTube consente a qualsiasi utente registrato di caricare contenuti, senza alcuna verifica della formazione professionale. I meccanismi di verifica presenti nella piattaforma, come la spunta blu introdotta per evitare furti di identità, certificano solo l'autenticità del canale, ma non la correttezza delle informazioni diffuse. Una ricerca [10] ha dimostrato come durante il periodo della pandemia SARS-CoV-2, una parte dei video più visionati in ambito medico-sanitario contenesse informazioni non corrette o false, evidenziando così i limiti di questo mo-

dello che diffonde informazioni riguardanti argomenti di carattere medico-scientifico che richiedono un'elevata affidabilità informativa.

Nel tentativo di eliminare tale problematica, YouTube nel 2021 ha introdotto l'iniziativa YouTube Health [11]: un tesserino identificativo visivo dedicato ai canali gestiti da organizzazioni sanitarie riconosciute, come i ministeri della salute, ospedali universitari e società scientifiche riconosciute. L'iniziativa, sebbene rappresenti un primo passo verso l'autenticazione delle fonti, presenta un limite rilevante ai fini di questo lavoro: la disponibilità geografica. Dal 2022 la candidatura è aperta anche ai singoli professionisti sanitari abilitati, oltre che alle organizzazioni, ma resta circoscritta a una dozzina di paesi fra i quali l'Italia non figura tuttora. Le stesse problematiche possono essere applicate anche a TikTok [12], dove l'algoritmo è orientato unicamente alla viralità del video diffondendo quindi contenuti sanitari brevi la cui correttezza non è sempre verificata.

2.2 Piattaforme specialistiche a pagamento

Una seconda categoria è rappresentata dalle piattaforme specialistiche a pagamento, rivolte principalmente a un pubblico di professionisti sanitari e accessibili solo tramite abbonamento. Un esempio è Streamed [13]: una piattaforma dedicata alla formazione di fisioterapisti, medici e figure simili. Questo servizio offre un abbonamento annuale di euro 598,00 e comprende il rilascio di crediti ECM previsti dalla normativa italiana sulla formazione continua obbligatoria [14]. I contenuti del servizio vengono scelti dagli amministratori e successivamente affidati a docenti qualificati, questo da una parte garantisce qualità del servizio, ma dall'altra esclude qualsiasi forma di produzione libera perché tutto è gestito e filtrato dalla piattaforma. Analogamente, Osmosis [15] si rivolge principalmente a studenti di medicina attraverso un catalogo di videolezioni pre-registrate, mentre Medscape [16] si concentra sull'aggiornamento professionale continuo per medici. Il costo di questi canali multimediali è una barriera all'accesso per l'utenza non specialistica orientata alla divulgazione medica.

2.3 Piattaforme accademiche e istituzionali

La terza categoria è costituita dalle piattaforme accademiche, quali Coursera [17], che è una piattaforma statunitense for-profit per la formazione online. Ospita corsi strutturati in ambito medico che vengono erogati in collaborazione con università di riferimento come la Johns Hopkins University o la Harvard University. Queste piattaforme si distinguono per l'organizzazione dei contenuti secondo il paradigma MOOC (Massive Open Online Courses), che prevede l'utilizzo sequenziale dei moduli, l'utilizzo di test di verifica e infine il rilascio della certificazione. Analoga struttura è offerta da edX [18]. Il modello è adatto alla formazione di contenuti specifici, ma non coincide con le esigenze dell'u-

tente non specialista, orientato alla consultazione di contenuti di carattere puramente divulgativo.

Un ulteriore esempio è MEDtube [19] che richiede l'accesso tramite verifica della qualifica dell'utente iscritto, per la condivisione peer-to-peer di video clinici tra professionisti sanitari.

2.4 Confronto tra le piattaforme

L'analisi eseguita ha dimostrato che nessuna delle piattaforme soddisfa tutti e quattro i requisiti individuati a partire dal problema descritto nel Capitolo 1: apertura al pubblico non specializzato, servizio gratuito, verifica delle qualifiche professionali del creator e libera pubblicazione dei contenuti da parte di professionisti autonomi. Il quadro comparativo delle piattaforme analizzate è riportato nella Tabella 2.1, che evidenzia la posizione di ciascuna soluzione rispetto alle dimensioni considerate e lo spazio nel quale si colloca NETMED.

Tabella 2.1: Confronto delle piattaforme più rappresentative di ciascuna categoria, rispetto ai quattro requisiti individuati.

Piattaforma	Pubblico	Costo	Verifica del creator	Pubblicazione autonoma
YouTube	Non specializzato	Gratuito	Solo identitaria	Sì
YouTube Health	Non specializzato	Gratuito	Qualifica professionale, in 12 paesi	Sì
TikTok	Non specializzato	Gratuito	Solo identitaria	Sì
Streamed	Professionisti	A pagamento	Docenti selezionati dalla piattaforma	No
Osmosis	Studenti di medicina	A pagamento	Autori selezionati dalla piattaforma	No
Coursera	Non specializzato	Gratuito o a pagamento	Docenti universitari	No
MEDtube	Professionisti	Gratuito con registrazione	Qualifica verificata all'iscrizione	Sì
NETMED	Non specializzato	Gratuito	Qualifica professionale	Sì

Tutte e tre le categorie presentano un limite strutturale che le rende inadatte al caso d'uso che si sta considerando. A differenza delle piattaforme generaliste, NETMED consente una verifica più approfondita delle qualifiche del creator rispetto alla semplice

conferma d'identità offerta dalla spunta blu. Nella piattaforma ogni utente che intende pubblicare contenuti è tenuto a dichiarare le proprie qualifiche professionali al momento della registrazione, fornendo informazioni relative al titolo, specializzazione e organizzazione di appartenenza. Queste richieste in un secondo momento vengono valutate dagli admin della piattaforma, i quali possono rifiutarle o approvarle. In seguito all'approvazione della richiesta, l'utente ottiene lo status di creator e l'autorizzazione per poter condividere i propri contenuti. Nel caso in cui l'utente pubblichi contenuti non idonei alla propria qualifica, l'amministratore può moderare i contenuti sia di propria iniziativa, sia in seguito a segnalazioni effettuate dagli utenti iscritti alla piattaforma. In questo modo la consultazione resta aperta a chiunque, mentre la possibilità di pubblicare rimane subordinata alla verifica della qualifica professionale.

A differenza delle applicazioni a pagamento, i contenuti sono totalmente liberi e gratuiti. Qualsiasi utente può visionare i video, iscriversi ai canali degli autori e partecipare ai dibattiti attraverso i commenti, tramite registrazione alla piattaforma senza bisogno di abbonamento. In questo modo la piattaforma risulta utilizzabile non solo dai professionisti ma anche dall'utente che vuole aumentare le proprie conoscenze in ambito sanitario tramite l'utilizzo di fonti verificate. Ne risulta un sistema aperto, che ha come scopo la divulgazione scientifica e che elimina ogni obbligo di pagamento per il servizio svolto, come invece accade nelle piattaforme specialistiche per il completamento del percorso e dell'obbligo formativo ECM.

I contenuti in NETMED sono organizzati per categoria e tag, e i video possono essere visualizzati singolarmente, proprio come accade nelle piattaforme di streaming come YouTube, nelle quali i contenuti sono visualizzabili da qualsiasi utente, a differenza delle piattaforme accademiche e istituzionali.

I creator che hanno pubblicato dei contenuti hanno la possibilità di segnalare commenti ritenuti non opportuni sotto i propri video. Le segnalazioni vengono valutate dall'amministratore. Se la segnalazione viene confermata, il commento è rimosso e il suo autore riceve uno strike. Al terzo strike l'account viene sospeso automaticamente in attesa di un intervento dell'amministratore, così da garantire una corretta e immediata gestione dei contenuti. La piattaforma implementa anche un sistema di notifiche, che avvisa gli utenti dei principali eventi che riguardano il loro profilo.

Si ottiene quindi un modello ibrido che combina l'apertura tipica delle piattaforme generaliste con la qualità dei contenuti delle piattaforme specialistiche, senza la necessità di acquistare un abbonamento, permettendo anche all'utente classico di accedere alla piattaforma. Le motivazioni tecniche e progettuali che hanno portato alla creazione della piattaforma e le tecnologie utilizzate per realizzarla sono descritte nel Capitolo 3 e nel Capitolo 4.

3

Tecnologie adottate

Il capitolo presenta le tecnologie utilizzate in NETMED e le ragioni per cui sono state scelte. All'interno di ogni ambito vengono descritte le diverse alternative considerate e i criteri che hanno portato a una determinata scelta. Le scelte qui motivate vengono applicate nel Capitolo 4.

3.1 Tecnologie per l'autenticazione web

L'autenticazione degli utenti è un aspetto centrale di qualsiasi applicazione che gestisce contenuti riservati e azioni sensibili. Il metodo di autenticazione condiziona l'intera architettura del sistema, la sua scalabilità e il livello di sicurezza che viene garantito all'utente finale. In questa sezione vengono descritte le principali soluzioni adottate nella piattaforma NETMED per l'autenticazione web e le motivazioni che hanno portato a queste scelte.

3.1.1 Autenticazione basata su sessione

Il modello classico dell'autenticazione web prevede l'utilizzo di una sessione lato server. Durante il login, il server verifica le credenziali di accesso dell'utente e genera un identificatore univoco della sessione, detto session ID. Tale identificatore viene inviato al client in un cookie, mentre lato server la sessione è conservata in una struttura dati persistente. Solitamente viene utilizzata una base di dati o una cache in memoria. A ogni

richiesta successiva, il browser rispedisce il cookie e il server verifica la validità dell'ID ricercandolo all'interno del proprio archivio.

Questa soluzione presenta un limite nelle applicazioni distribuite: prevede che il server debba mantenere in memoria lo stato di ogni sessione attiva. Questo indica la necessità di introdurre dei meccanismi di sincronizzazione tra più istanze del server nel caso di deployment orizzontale. La soluzione introduce complessità architetturale e limita la capacità del sistema di scalare.

3.1.2 Autenticazione basata su token: JSON Web Token

Un diverso approccio è quello dell'autenticazione basata su Token che si è diffuso durante l'affermazione delle architetture RESTful e delle Single Page Application. In questo modello, il server non va a memorizzare lo stato della sessione ma restituisce all'utente autenticato un token firmato, che il client conserva localmente e che ripresenta al server durante le successive richieste. Il server verifica la correttezza del token solo tramite la firma.

Lo standard utilizzato per la rappresentazione di tali token è il JSON Web Token (JWT) [20]. Un JWT è costituito da tre segmenti principali: header, payload e firma. L'header contiene le informazioni sull'algoritmo di firma utilizzato, il payload contiene coppie chiave-valore che descrivono l'utente autenticato (id, ruolo, scadenza del token). La firma garantisce l'integrità del token e ne impedisce la falsificazione.

La natura stateless del token JWT presenta due vantaggi. In primo luogo, elimina la necessità di memorizzare lo stato lato server, semplificando la scalabilità orizzontale del sistema: qualunque istanza del server può verificare autonomamente la validità del token senza necessità di consultare l'archivio condiviso. In secondo luogo, disaccoppia il backend dai client, rendendo i token utilizzabili anche da piattaforme mobili o da servizi di terze parti che si integrano con le API.

3.1.3 Firma del token: HMAC-SHA256

La firma di un JWT può essere calcolata usando algoritmi simmetrici o asimmetrici. L'algoritmo simmetrico più diffuso è HMAC-SHA256, indicato negli header JWT con la sigla HS256 [21]. HMAC [22], acronimo di Hash-based Message Authentication Code, è una costruzione crittografica che combina la funzione di hash con una chiave segreta, per produrre un codice di autenticazione del messaggio.

La firma HMAC-SHA256 garantisce due proprietà fondamentali: l'integrità del token, poiché qualsiasi modifica effettuata al payload del pacchetto porterebbe a una firma non valida, e l'autenticità del token, in quanto solo chi possiede la chiave segreta può generarne uno valido. È importante sottolineare che HS256 non cifra il payload, che

rimane leggibile in chiaro: per questa ragione all'interno di un token JWT non devono mai essere presenti informazioni sensibili.

3.1.4 Hashing sicuro delle password

Un ulteriore aspetto critico dell'autenticazione riguarda il modo in cui vengono salvate le password degli utenti registrati. Le password non devono essere salvate in chiaro nel database, in quanto un accesso non autorizzato porterebbe alla compromissione immediata degli account utente. La pratica consolidata prevede la memorizzazione del solo hash della password, ottenuto tramite l'applicazione di una funzione crittografica non invertibile.

L'utilizzo di funzioni di hash generiche come SHA-256 [23] è sconsigliato nel caso specifico di protezione delle password, in quanto la loro rapidità di calcolo consente ad un attaccante che entri in possesso della base di dati di eseguire attacchi di brute force. Per evitare queste problematiche sono state introdotte delle apposite funzioni di hashing computazionalmente costose, note come funzioni di hashing adattivo.

La funzione bcrypt [24] è tra le soluzioni più diffuse in tale ambito. Integra al proprio interno un salt casuale, rendendo inefficaci gli attacchi basati su tabelle precomputeate, e un parametro dinamico detto cost factor, che determina il numero di iterazioni interne dell'algoritmo. Aumentando il cost factor si incrementa in modo esponenziale il tempo necessario per il calcolo di un singolo hash, adattando in modo dinamico la funzione all'evoluzione della potenza di calcolo disponibile. All'interno di NETMED viene utilizzato un valore di cost pari a 10, corrispondente a $2^{10} = 1024$ iterazioni della fase di espansione della chiave. La misurazione riportata nella Sezione 5.3.3 indica un tempo medio di 43,9 ms per hash sull'hardware di prova. Un valore di questo ordine rende impraticabile la ricerca esaustiva su tutto lo spazio delle password, mentre un attacco a dizionario su credenziali deboli resta possibile: per questa ragione la registrazione applica anche una politica di robustezza sulla password, descritta nella Sezione 4.9.4.

Un'alternativa a bcrypt è Argon2 [25], vincitrice della Password Hashing Competition e raccomandata dall'OWASP [26] come funzione di prima scelta. Argon2 introduce, oltre al costo computazionale, anche un costo in termini di memoria, rendendo più costosi gli attacchi condotti tramite hardware specializzato come le GPU. La scelta di bcrypt al posto di Argon2 nel progetto è stata dettata dal maggior numero di librerie disponibili nell'ecosistema Node.js e dall'ampia disponibilità di documentazione offerta.

3.1.5 Scelta adottata in NETMED

Tali considerazioni sulla sicurezza e sulla scalabilità hanno contribuito alla decisione di utilizzare in NETMED l'autenticazione basata su JWT con firma HMAC-SHA256, in

combinazione con la conservazione delle password attraverso l'hashing con la funzione bcrypt configurata con cost factor pari a dieci.

La combinazione di queste scelte permette di ottenere un sistema stateless, adatto ad un'eventuale futura evoluzione verso un'architettura distribuita, garantendo però anche un livello di sicurezza che rispecchia le raccomandazioni correnti in materia di autenticazione web.

3.2 Persistenza dei dati

La gestione dei dati persistenti è una questione centrale in qualsiasi applicazione web. In base al modo in cui viene gestita la base di dati, ci saranno cambiamenti nelle prestazioni delle interrogazioni, sulla coerenza dei dati in presenza di operazioni concorrenti e sulla facilità di manutenzione dello schema durante l'evoluzione del progetto. Nel corso degli ultimi due decenni si sono affermate due categorie in particolare: le basi di dati relazionali [27] e le basi di dati NoSQL sviluppate per rispondere alle esigenze di scalabilità orizzontale delle piattaforme web di grandi dimensioni.

3.2.1 Basi di dati relazionali

Le basi di dati relazionali organizzano le informazioni in tabelle; queste sono collegate fra di loro attraverso delle chiavi primarie ed esterne. Ciascuna tabella è un'entità del dominio applicativo e le relazioni tra le diverse tabelle vengono espresse mediante dei vincoli referenziali. L'interrogazione della base di dati avviene tramite il linguaggio SQL, standardizzato dall'ISO. All'interno dei sistemi relazionali vengono rispettate le proprietà ACID (Atomicity, Consistency, Isolation, Durability) [28], che garantiscono l'integrità dei dati anche in presenza di transazioni concorrenti.

3.2.2 Basi di dati NoSQL

Le basi di dati NoSQL rinunciano in modo parziale o totale al modello relazionale in favore di modelli più flessibili come documenti (MongoDB) e coppie chiave-valore (Redis). Questi sistemi offrono maggiore scalabilità orizzontale e permettono di avere uno schema flessibile, per effettuare modifiche alla struttura dati senza eseguire migrazioni complesse. Tale beneficio comporta una riduzione delle garanzie ACID, che vengono sostituite dal modello BASE (Basically Available, Soft state, Eventual consistency) [29], meno stringente in termini di coerenza.

3.2.3 PostgreSQL e il tipo di dato JSONB

Per la realizzazione della piattaforma di streaming NETMED si è scelto di adottare PostgreSQL [30] che è un sistema di gestione di basi di dati relazionali open source. La scelta è stata fatta per tre fattori principali: il primo riguarda il pieno rispetto delle proprietà ACID, per garantire la coerenza di operazioni che coinvolgono più tabelle, come l'inserimento di un video con i relativi tag o l'approvazione di una richiesta di verifica, che aggiorna il profilo dell'utente, genera una notifica e registra l'azione nel registro amministrativo. Il secondo è la maturità del sistema che offre driver stabili per Node.js e una documentazione ampia. Il terzo è il supporto al tipo di dato JSONB [31],

che consente di memorizzare documenti JSON direttamente nelle colonne relazionali, avendo così i vantaggi di entrambi i paradigmi.

Il tipo JSONB è stato utilizzato all'interno della piattaforma per la memorizzazione del payload delle notifiche e dei profili verificati dei creator, in cui la struttura del contenuto varia in base all'evento. Un'alternativa sarebbe stata definire una tabella distinta per ciascun tipo di payload; grazie a JSONB è invece sufficiente un'unica tabella.

3.2.4 Applicazione a NETMED

All'interno di NETMED è stato utilizzato PostgreSQL poiché sono presenti relazioni ben definite tra le varie entità. Queste relazioni sono rappresentabili tramite chiavi esterne e vincoli di integrità referenziale che vengono garantiti da una base di dati relazionale. Allo stesso tempo, la presenza del tipo JSONB ha permesso di soddisfare i pochi casi d'uso caratterizzati da schema variabile senza dover introdurre un secondo sistema di persistenza. Nel Capitolo 4 sono descritte le principali tabelle dello schema e le motivazioni progettuali.

3.3 Architetture front-end

Lo sviluppo delle interfacce web negli ultimi anni ha visto la transizione dalle architetture tradizionali basate su pagine multiple verso modelli più dinamici in cui l'intera applicazione risiede lato client. Si sono così affermati due modelli, entrambi con vantaggi e svantaggi a seconda dell'uso. In questa sezione vengono descritti i criteri che orientano la scelta fra un approccio Multi-Page Application e una Single Page Application.

3.3.1 Multi-Page Application

Il modello Multi-Page Application (MPA) è l'architettura utilizzata fin dagli inizi del World Wide Web. In questo modello ciascuna pagina corrisponde a un file HTML diverso, generato dal server in risposta a una richiesta HTTP. La navigazione fra le pagine porta a un caricamento completo di un nuovo documento, con la ricostruzione dell'intero albero DOM da parte del browser. L'interattività delle pagine è affidata a JavaScript, che gestisce i comportamenti della pagina, quali validazione dei form, animazioni oppure richieste asincrone.

3.3.2 Single Page Application

Il modello Single Page Application (SPA) si basa su un principio completamente diverso: l'intera applicazione viene servita al browser come un unico documento HTML, integrato tramite un insieme di file JS che si occupano di aggiornare dinamicamente il DOM in base alle varie interazioni dell'utente. Il server si limita unicamente a fornire i dati in formato JSON attraverso un'interfaccia API, senza generare nuove pagine HTML.

Il modello SPA è supportato da framework moderni come React, Vue e Angular, ciascuno dei quali introduce una propria astrazione per la gestione dello stato, il rendering condizionale dei componenti e il routing interno dell'applicazione. Questo approccio garantisce all'utente un'esperienza fluida in quanto le transizioni tra le varie schermate non richiedono l'aggiornamento della pagina e favoriscono una divisione tra logica di presentazione e logica di dominio.

Questo approccio però possiede dei limiti. Il bundle JavaScript risultante dalla compilazione dei framework ha dimensioni elevate e questo ha un impatto significativo sul tempo di primo caricamento [32] soprattutto su dispositivi mobili e su connessioni lente. Inoltre, per l'utilizzo di queste tecnologie è richiesta una conoscenza approfondita del framework adottato.

La scelta fra i due modelli è stata fatta in base al tipo di interazione previsto. Le pagine rivolte all'utente seguono il modello MPA, poiché la navigazione avviene fra contenuti indipendenti l'uno dall'altro. Il pannello amministratore è realizzato come Single Page Application: l'amministratore alterna di continuo elenchi diversi nello stesso

contesto di lavoro, e un ricaricamento della pagina avrebbe reso l'uso più lento senza alcun vantaggio.

3.3.3 JavaScript vanilla e scelta adottata in NETMED

Un terzo approccio, meno diffuso ma tuttora valido, è l'utilizzo del front-end senza il ricorso a framework esterni, utilizzando esclusivamente le API standard del browser e il linguaggio JavaScript. Questo approccio permette di ridurre il peso del codice inviato all'utente, di eliminare le dipendenze da librerie esterne e di rendere trasparente ogni singola istruzione eseguita dal browser. NETMED adotta questa soluzione: una singola pagina della piattaforma corrisponde a un file HTML. La motivazione principale è la leggibilità del codice: l'assenza di un framework garantisce che ogni comportamento del client sia leggibile senza doverne comprendere le astrazioni. Un ulteriore vantaggio riguarda l'assenza di un processo di build: le pagine HTML e JS vengono fornite direttamente al browser senza fasi intermedie di compilazione, semplificando l'attività di debugging e riducendo la possibilità di errore.

3.4 Containerizzazione

Il deployment di un'applicazione web comporta la necessità di configurare l'ambiente di lavoro del server in modo coerente con quello utilizzato durante lo sviluppo così che l'applicazione funzioni su dispositivi diversi. In caso contrario le differenze di comportamento sono attribuibili alla versione del sistema operativo, del runtime oppure delle librerie di sistema. La containerizzazione è la soluzione a questa problematica.

3.4.1 Il modello Docker

Docker [33] è un motore di containerizzazione che consente di impacchettare un'applicazione e tutte le sue dipendenze all'interno di un artefatto immutabile detto immagine, eseguibile su qualsiasi sistema che disponga del runtime Docker; l'esecuzione di un'immagine crea un container. A differenza delle macchine virtuali tradizionali, che virtualizzano l'intero sistema operativo, i container Docker condividono con l'host il kernel Linux, o quello di una macchina virtuale gestita dal runtime nel caso di host Windows e macOS, e isolano l'esecuzione delle applicazioni.

Un'immagine Docker viene descritta attraverso un file di testo denominato Dockerfile, che specifica tutte le istruzioni necessarie per costruirla: l'immagine di base, i file da copiare al suo interno, i comandi di installazione delle dipendenze e il comando di avvio del processo principale. Il risultato è un artefatto immutabile, composto da una sequenza di livelli ciascuno generato da una o più istruzioni del Dockerfile che possono essere condivisi tra più immagini per evitare la duplicazione dei dati e ridurre sia lo spazio di archiviazione richiesto sia i tempi di trasferimento delle immagini.

3.4.2 Docker Compose

Un'applicazione web moderna è composta da più processi che devono essere avviati insieme. In NETMED, ad esempio, la piattaforma richiede l'esecuzione simultanea di un server applicativo Node.js [34] e di un'istanza del database PostgreSQL che devono comunicare fra di loro. La gestione manuale dei container sarebbe più complessa e soggetta ad ampi margini di errore.

Docker Compose [35] è lo strumento di gestione locale di applicazioni multi-container. Tramite un file denominato `docker-compose.yml`, vengono descritti i servizi che compongono l'applicazione, le relazioni di dipendenza, le porte esposte all'host, i volumi persistenti e le variabili d'ambiente. Un unico comando, `docker compose up`, si occupa di costruire tutte le immagini necessarie, di avviare i container nell'ordine corretto e di configurare la rete interna per la comunicazione.

3.4.3 Ambiente di sviluppo

Per la realizzazione della piattaforma è stato utilizzato Docker Compose per orchestrare i due principali servizi: un container denominato `netmed-app` che esegue il server applicativo Node.js e un container denominato `netmed-db` che esegue l'istanza del database PostgreSQL. La comunicazione avviene tra i due container tramite una rete Docker dedicata, mentre la porta 3000 del container applicativo viene esposta all'esterno per consentire l'accesso da parte del browser. I dati del database sono persistenti in un volume Docker, che ne preserva il contenuto anche in caso di riavvii o ricostruzioni dei container.

La motivazione principale per l'adozione di un ambiente Docker risiede nella riproducibilità dell'ambiente di esecuzione. Chiunque voglia replicare il progetto può clonare il repository e avviare l'intera piattaforma con un unico comando, senza dover installare Node.js o PostgreSQL nel proprio ambiente né configurare manualmente la connessione tra i due sistemi. Un secondo vantaggio riguarda la portabilità del progetto verso ambienti di produzione o servizi di hosting containerizzati.

3.5 Hosting dei contenuti video

I video presenti su NETMED non sono archiviati nella piattaforma: ciascun contenuto viene riprodotto attraverso il player di YouTube a partire dal proprio identificativo memorizzato nella base di dati. Alla base della scelta c'è un vincolo concreto. Non è stato possibile disporre di materiale video originale, poiché i contenuti di ambito clinico pongono problemi di tutela dei dati personali. Si è quindi optato per l'integrazione dei contenuti già pubblicamente accessibili. La critica rivolta nel Capitolo 2 alle piattaforme generaliste riguarda l'assenza di un controllo sulla qualifica di chi pubblica, non il servizio di distribuzione in sé. NETMED interviene sul primo piano, ma lascia il secondo a un servizio esterno. Il problema di questa architettura è la dipendenza da un fornitore terzo: se un video viene rimosso da YouTube, sulla piattaforma resta un riferimento non più riproducibile, e NETMED non ha modo di ripristinarlo.

4

Progettazione e implementazione

In questo capitolo vengono descritte la progettazione e l'implementazione di NETMED. Le prime sezioni presentano l'architettura del sistema, il modello dei dati e la rappresentazione tramite la modellazione UML e i sequence diagram principali. Seguono la struttura del backend, la progettazione dell'API REST, e quella del frontend. Le ultime tre sezioni discutono della verifica del creator e del sistema di notifiche, e riportano l'implementazione dei componenti più significativi.

4.1 Architettura generale del sistema

NETMED è stato progettato seguendo il paradigma client-server, a tre livelli, che separa in modo netto il livello di presentazione da quello applicativo e di persistenza. Questa divisione permette di rendere indipendenti le varie componenti, semplifica il debug e consente la futura scalabilità del sistema. In Figura 4.1 viene raffigurata la panoramica dei componenti principali e le relative relazioni.

4.1.1 Componenti principali

Come mostrato nella Figura 4.1, il sistema è formato da tre componenti principali. Il client è costituito da un insieme di pagine HTML servite come contenuto statico dal server, ciascuna delle quali carica i propri fogli di stile CSS e gli script JavaScript. Il codice lato client viene interpretato direttamente dal browser dell'utente. La logica lato

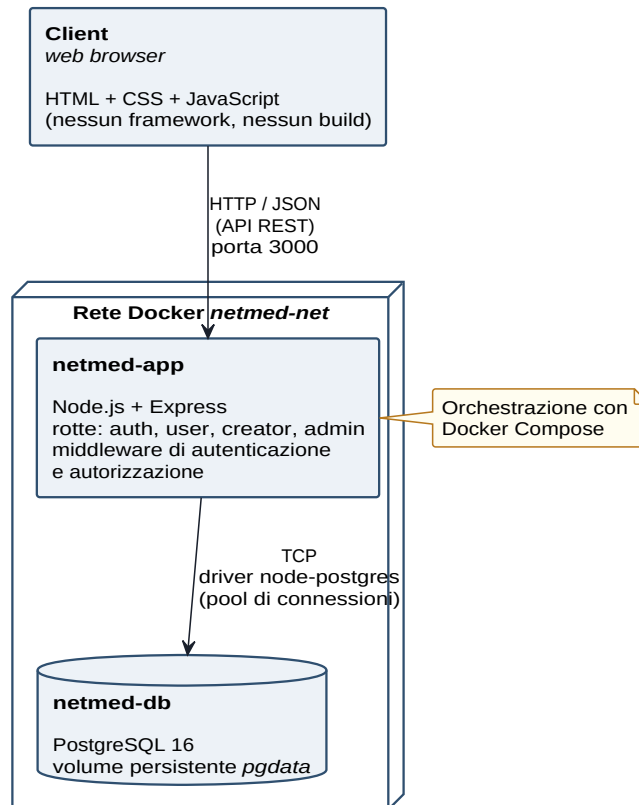


Figura 4.1: Architettura a tre livelli di NETMED. Il client comunica con il server tramite il protocollo HTTP; il server accede al database tramite una connessione TCP. Tutto avviene all'interno di una rete Docker.

client si occupa della gestione dell'interfaccia utente, della ricezione delle interazioni e della gestione delle richieste asincrone al server.

Il server applicativo è il componente centrale di NETMED e viene eseguito nel container `netmed-app`. Dentro questo container l'applicazione Node.js [34] utilizza il framework Express [7] per esporre due tipologie di risorse: il contenuto statico del frontend, dentro la cartella `public/`, e un insieme di endpoint REST organizzati in quattro moduli diversi divisi in base al ruolo (autenticazione, utente comune, creator e amministratore). Le richieste dirette alle risorse protette attraversano dei middleware che verificano la validità del token JWT e il ruolo dell'utente.

Il livello di persistenza è costituito da un'istanza di PostgreSQL 16 [30] eseguita nel container `netmed-db`. Il server applicativo comunica con la base di dati tramite il driver `pg` [36], che gestisce un pool di connessioni condiviso tra tutte le richieste in ingresso. In questo modo si evita la chiusura e l'apertura della connessione ad ogni

singola query. Nella Sezione 4.2 sono descritte le principali tabelle dello schema e le motivazioni progettuali di ciascuna di esse.

4.1.2 Flusso di una richiesta

Per assicurare una corretta comprensione dell'architettura della piattaforma, in questo paragrafo viene descritto il ciclo di vita di una richiesta HTTP diretta al server. Si prende ad esempio la richiesta effettuata dal client, quando un utente aggiunge un video ai suoi preferiti.

Tramite un metodo POST il browser dell'utente costruisce una richiesta HTTP verso l'endpoint `/api/user/videos/:id/favorite`, inserendo nell'header `Authorization` il token JWT ricevuto durante il login. La richiesta viene presa in gestione dal container `netmed-app`, dove il server Express si occupa di inoltrare la richiesta al modulo delle rotte utente. Prima che la richiesta raggiunga l'effettivo destinatario, questa attraversa il middleware `authenticateToken`, che verifica la firma del token utilizzando l'algoritmo HMAC-SHA256. Il gestore della rotta verifica se il video sia già presente tra i preferiti dell'utente, e in base all'esito, esegue una query di inserimento o cancellazione sulla tabella `video_favorites`: se l'operazione va a buon fine il server risponde con un codice 200 OK e un oggetto JSON che riporta lo stato aggiornato del preferito.

Il flusso descritto è simile per la maggior parte degli endpoint utilizzati nella piattaforma: variano il metodo HTTP, la rotta di destinazione e la logica applicativa, ma la struttura generale e l'autenticazione tramite middleware rimangono invariate.

4.1.3 Ambiente di esecuzione

Come descritto nella precedente Sezione 3.4, l'intera piattaforma viene orchestrata mediante Docker Compose [35], dove sono presenti due principali container `netmed-app` e `netmed-db` che condividono una rete Docker dedicata, isolata dall'host. La porta 3000 del container viene esposta verso l'host, consentendo di raggiungere l'applicazione dal browser usando l'indirizzo `http://localhost:3000`. I dati del database sono persistenti su un volume Docker per preservare i dati anche in caso di riavvio dei container. La configurazione dell'intero stack viene definita nel file `docker-compose.yml`.

Un ulteriore vantaggio di questa configurazione è dato dal livello di automazione del sistema al primo avvio. Nel server applicativo è presente una funzione, denominata `seedAdmin()`, che verifica l'esistenza di un utente amministratore nella base di dati, nel caso in cui non fosse presente, ne crea uno, utilizzando delle credenziali predefinite. In modo simile anche lo schema della base di dati viene creato automaticamente al primo avvio del container `netmed-db` tramite l'utilizzo di uno script SQL. In questo modo chiunque sia in possesso del repository, può utilizzare la piattaforma senza necessità di configurare il sistema.

4.1.4 Pool di connessioni

L'accesso alla base di dati PostgreSQL avviene tramite il driver ufficiale node-postgres, configurato secondo il pattern del connection pool. Il modulo `db/db.js` istanzia al bootstrap dell'applicazione un oggetto `Pool` e lo esporta come singleton condiviso da tutti i gestori delle rotte.

Il pool mantiene connessioni già aperte verso il database, che vengono assegnate su richiesta ai gestori delle rotte al momento dell'esecuzione di un'interrogazione, e al termine della query, la connessione viene rilasciata al pool. Questa modalità consente di ridurre il costo di apertura di una nuova connessione per ciascuna richiesta HTTP, costo che altrimenti sarebbe composto dal three-way handshake TCP, dall'autenticazione al database e dall'inizializzazione della sessione. La scelta di adottare un pool invece di un modello a connessione singola condivisa è data dalla natura non bloccante di Node.js, poiché il pool garantisce connessioni fisiche distinte in modo trasparente ai gestori della rotta.

4.2 Modello dei dati

Le tabelle della base di dati di NETMED sono raggruppate in cinque famiglie principali: utenti e verifica professionale, catalogo dei contenuti, interazioni degli utenti, moderazione e amministrazione, sistema delle notifiche.

La Figura 4.2 riporta lo schema entità-relazione con le principali relazioni tra le tabelle.

tabella avrebbe richiesto la creazione di diverse colonne opzionali. Tramite l'utilizzo di JSONB è possibile mantenere una struttura pulita, garantendo anche la possibilità di effettuare interrogazioni alla base di dati.

4.2.2 Contenuti e catalogazione

La tabella `videos` è la componente centrale dello schema di NETMED, ciascuna riga rappresenta un video pubblicato nella piattaforma. Il video non viene memorizzato fisicamente nel sistema, ma viene utilizzato un lettore YouTube: la tabella tra i vari parametri contiene l'identificativo YouTube del video (`youtube_id`), il titolo, la descrizione, il riferimento all'autore del video (`uploaded_by`), la categoria di appartenenza (`category_id`), un flag `is_flagged` che indica se il video è stato bloccato in seguito a segnalazione. I conteggi delle visualizzazioni, dei like e dei commenti non sono memorizzati nella tabella `videos`, ma vengono calcolati al momento dell'interrogazione mediante una sottoquery sulle tabelle di dettaglio. La scelta garantisce la coerenza del dato: non esistono contatori da mantenere allineati e nessuna operazione di scrittura può lasciare il sistema in uno stato inconsistente. Il costo è a carico della lettura ed è sostenibile alle dimensioni attuali del catalogo. La classificazione avviene tramite l'utilizzo di due tabelle: la tabella `categories` che contiene le categorie tematiche del catalogo ed è collegata alla tabella `videos` tramite una relazione 1:N. La tabella `tags` contiene i tag testuali che è possibile associare al video, ed è modellata tramite una relazione N:N con la tabella `videos`, realizzata tramite la tabella di associazione `video_tags`. Durante l'inserimento dei tag, vengono normalizzati in minuscolo tramite la funzione `toLowerCase` così da evitare duplicati.

4.2.3 Interazione degli utenti

Sono presenti sette tabelle principali che si occupano dell'interazione tra gli utenti. La tabella `comments` contiene i contenuti pubblicati sotto i video, ciascuna riga fa riferimento al video commentato e all'autore, contiene il testo del commento e un campo opzionale `parent_id` che consente di annidare i commenti in una struttura ad albero in caso di risposta a un altro commento. La tabella `likes` si occupa di memorizzare i like degli utenti nei confronti del video, mediante una coppia (`video_id`, `user_id`) sulla quale è definito un vincolo di unicità che impedisce di esprimere più di una valutazione sullo stesso video. La tabella `video_favorites` memorizza i video salvati dall'utente nei propri preferiti, dove è anche possibile creare un'organizzazione tramite delle cartelle rappresentate dalle tabelle `collections` e `collection_videos`. La tabella `nm_user_follows` modella le iscrizioni tra un utente e un creator, mentre la tabella `views` permette di registrare la cronologia dei video visualizzati da ciascun utente insieme al numero di visualizzazioni e alla data dell'ultimo accesso del contenuto.

4.2.4 Moderazione e amministrazione

La moderazione è affidata alla tabella `reports`, alla tabella `comment_reports` e alla colonna `strike_count` della tabella `users`; il tracciamento delle azioni amministrative è affidato alla tabella `admin_audit`. La tabella `reports` contiene le segnalazioni prodotte dagli utenti di fronte a contenuti ritenuti inappropriati. Ciascuna segnalazione fa riferimento a un determinato video, all'utente che l'ha effettuata, alla motivazione di tale scelta, a un commento libero e all'eventuale stato di lavorazione della richiesta. Il contatore `strike_count` memorizza gli strike accumulati dall'autore dei commenti rimossi, in seguito alle segnalazioni. Al raggiungimento del terzo strike l'utente verrà automaticamente bannato, mettendo il ruolo dell'utente a `banned`, fino a un intervento dell'amministratore. La tabella `admin_audit` si occupa delle azioni amministrative significative (approvazioni di verifiche, ban utenti), tenendo traccia in ogni momento di chi ha effettuato queste operazioni e quando.

4.2.5 Sistema delle notifiche

In NETMED è presente un sistema di notifiche e questo viene gestito tramite la tabella `notifications`, che memorizza le notifiche dirette a ciascun utente. Ogni riga contiene il riferimento all'utente destinatario, con un campo `type` che indica la tipologia dell'evento, un campo `read_at` che indica se la notifica è stata letta, e un campo di tipo JSONB denominato `payload` che contiene diverse informazioni in base alla tipologia di notifica.

L'inserimento di una notifica è effettuato dal server in modalità best-effort, per garantire la miglior esperienza all'utente. Nel caso in cui l'inserimento fallisse per problemi legati al database, l'operazione principale a cui la notifica era collegata non viene interrotta.

4.3 Modellazione UML delle entità principali

Il diagramma di Figura 4.3 non riflette una struttura orientata agli oggetti: il backend è organizzato in moduli di routing e non in classi. Le entità rappresentano le tabelle descritte nella Sezione 4.2; le operazioni indicano i comportamenti implementativi nei rispettivi gestori.

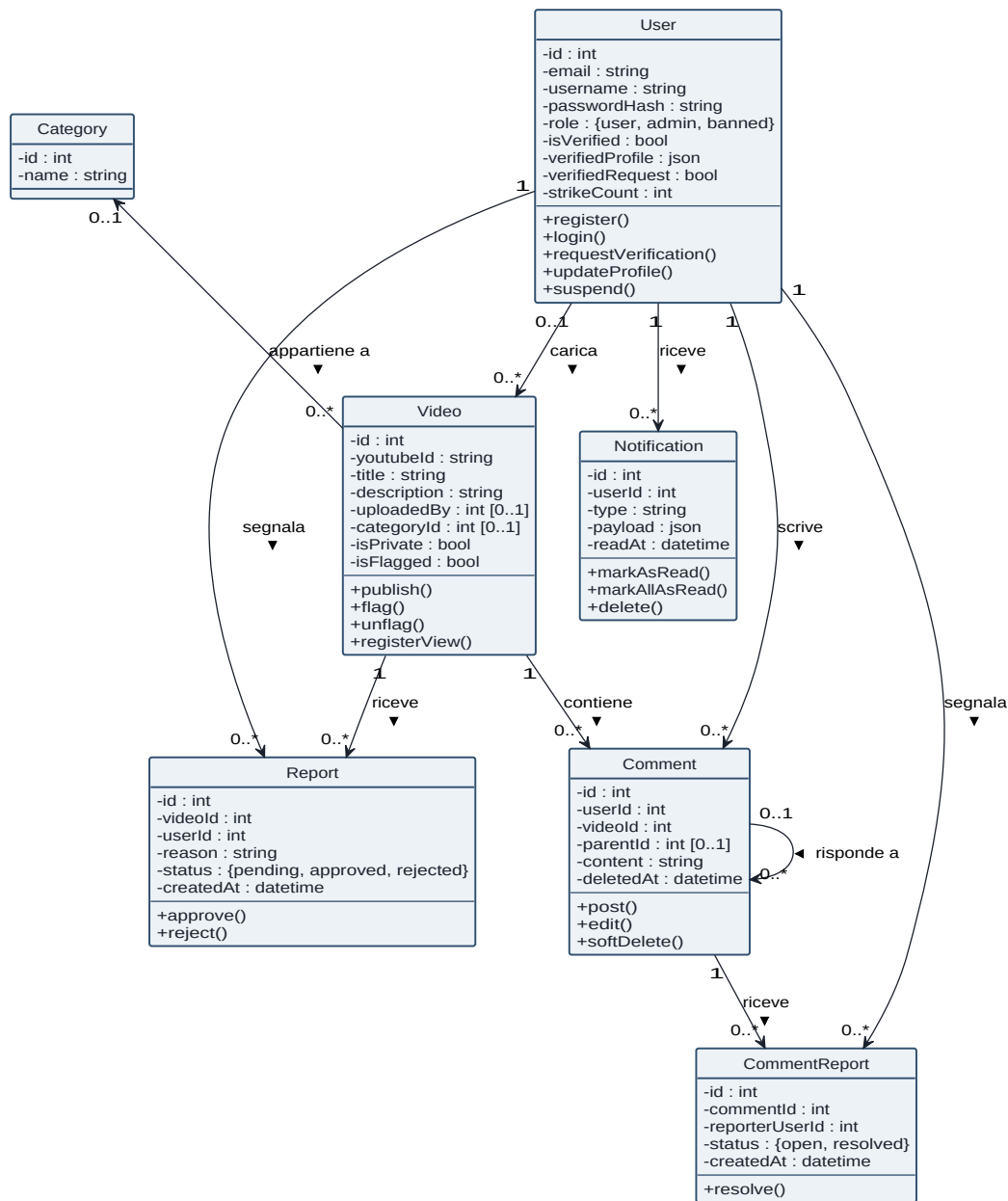


Figura 4.3: Diagramma delle classi del dominio applicativo.

4.3.1 Classi del dominio

La classe **User** è l'utente registrato alla piattaforma, senza necessità di specificare il ruolo. I principali attributi di questa classe sono l'id, le credenziali di accesso, il ruolo e

il flag `isVerified`. Le operazioni principali di questa classe sono `register()`, `login()`, `requestVerification()` e `updateProfile()`.

La classe `Video` rappresenta un singolo video della piattaforma. Il video, come descritto nella Sezione 4.2.2, non viene memorizzato fisicamente. È visionato tramite un lettore YouTube, e l'entità ne contiene l'identificativo insieme alle informazioni principali. I metodi di tale entità sono `publish()`, `flag()`, `unflag()` e `registerView()`.

La classe `Comment` rappresenta un commento pubblicato da un utente sotto a un video. La struttura ad albero viene realizzata mediante l'utilizzo di un attributo `parentId`, per collegare ciascun commento al padre. Questo permette di leggere i commenti di risposta in modo strutturato.

La classe `Report` indica una segnalazione effettuata da un utente nei confronti di un contenuto ritenuto inappropriato. I suoi attributi principali sono la motivazione e lo stato di lavorazione della segnalazione. Le operazioni più significative di questa classe sono `approve()` e `reject()`, entrambe sono riservate esclusivamente all'amministratore.

La classe `Notification` è l'entità che notifica gli utenti della piattaforma. Ciascuna notifica contiene un riferimento al destinatario, un tipo di evento, un payload in formato JSONB e un campo `readAt` che ne indica lo stato di lettura. Le operazioni principali eseguibili da questa classe sono `markAsRead()`, `markAllAsRead()` e `delete()`.

4.3.2 Relazioni tra le entità

Le entità sopra rappresentate formano il grafo del dominio applicativo. Un utente può caricare zero o più video, tramite l'attributo `uploadedBy`; può pubblicare zero o più commenti e può segnalare più contenuti. La relazione ricorsiva eseguita su `Comment` consente di rappresentare le repliche annidate: ciascun commento può avere un unico commento padre, ma può avere più commenti figli. L'insieme delle relazioni descritte permette di ottenere un sistema efficiente in grado di rispondere alle interrogazioni che vengono eseguite nella piattaforma, quali ad esempio “quali sono i video pubblicati da un creator” e “le segnalazioni in attesa di valutazione”. Nella Sezione 4.2 sono state descritte le corrispondenti implementazioni SQL, che utilizzano delle chiavi esterne.

4.4 Sequence diagram dei flussi chiave

Per descrivere al meglio le interazioni che avvengono tra i vari componenti del sistema durante l'esecuzione dei principali casi d'uso, sono stati creati tre sequence diagram. Ciascun diagramma mostra le azioni e i messaggi che vengono scambiati fra i vari attori coinvolti nei flussi significativi della piattaforma: l'autenticazione dell'utente (Figura 4.4), la richiesta di verifica del profilo creator (Figura 4.5) e la segnalazione di un contenuto (Figura 4.6).

4.4.1 Flusso di autenticazione dell'utente

Il login è l'operazione più importante effettuata nella piattaforma e costituisce il presupposto per l'accesso alle funzionalità riservate agli utenti registrati del sistema. La sequenza dei messaggi scambiati è illustrata in Figura 4.4.

Il flusso viene gestito in due fasi. Il browser invia una richiesta HTTP all'endpoint `/api/auth/login`: questa richiesta contiene l'indirizzo e-mail e la password indicati dall'utente nel corpo JSON. Il middleware `rateLimit` verifica il numero di tentativi di autenticazione effettuati e, in caso di superamento della soglia, restituisce direttamente al client un codice HTTP "429 Too Many Requests". Se il controllo viene passato, il gestore della rotta esegue una query sulla tabella `users` per verificare l'e-mail. Nel caso non venisse trovata, il server restituisce un codice 401 Unauthorized, con un messaggio generico che non distingue tra indirizzo inesistente e password errata, così da non rivelare quali e-mail sono registrate sulla piattaforma [37]. In caso di corrispondenza, il gestore invoca la funzione `bcrypt.compare` per confrontare la password ricevuta con l'hash memorizzato nella base di dati. Se tutto va a buon fine il server costruisce il payload del token JWT [20] che include l'id, l'indirizzo e-mail, lo username, il ruolo, il flag `is_verified` e la data di scadenza, e lo firma con la chiave segreta tramite l'algoritmo HMAC-SHA256 [21]. Il token risultante viene restituito al client all'interno di una risposta "200 OK", insieme ai dati non sensibili dell'utente. Il browser memorizza il token nel `localStorage` e lo utilizza in ogni successiva richiesta, inserendolo nell'header [38].

4.4.2 Richiesta di verifica del profilo creator

Per garantire il controllo delle qualifiche del creator, il processo di verifica è diviso in due fasi: la fase di richiesta, effettuata dall'utente al momento della registrazione o successivamente dalla propria pagina utente, e la fase di valutazione, nella quale l'amministratore, tramite il pannello dedicato, deciderà se accettare o rifiutare tale richiesta. Il sequence diagram in Figura 4.5 illustra entrambe le fasi.

Nella prima fase, l'utente compila il form con tutte le informazioni richieste per diventare creator. Il browser invia una richiesta POST all'endpoint `/api/user/me/request-creator` con il payload strutturato in formato JSON. Dopo che il server ha verificato il token dell'utente, esegue un update sulla tabella `users`, che valorizza il campo `verified_profile` con l'oggetto JSON ricevuto e imposta il campo `verified_request_at`. L'utente non diventa immediatamente creator, ma la richiesta viene registrata e resa disponibile nel pannello amministratore.

Nella seconda fase, l'amministratore accede all'elenco e visualizza le richieste in attesa mediante una chiamata all'endpoint `GET /api/admin/creator-requests`. Selezionato l'utente da approvare, l'admin invia una richiesta POST all'endpoint `/api/admin/verified-requests/:uid/approve`. Il server successivamente verifica il ruolo effettivo

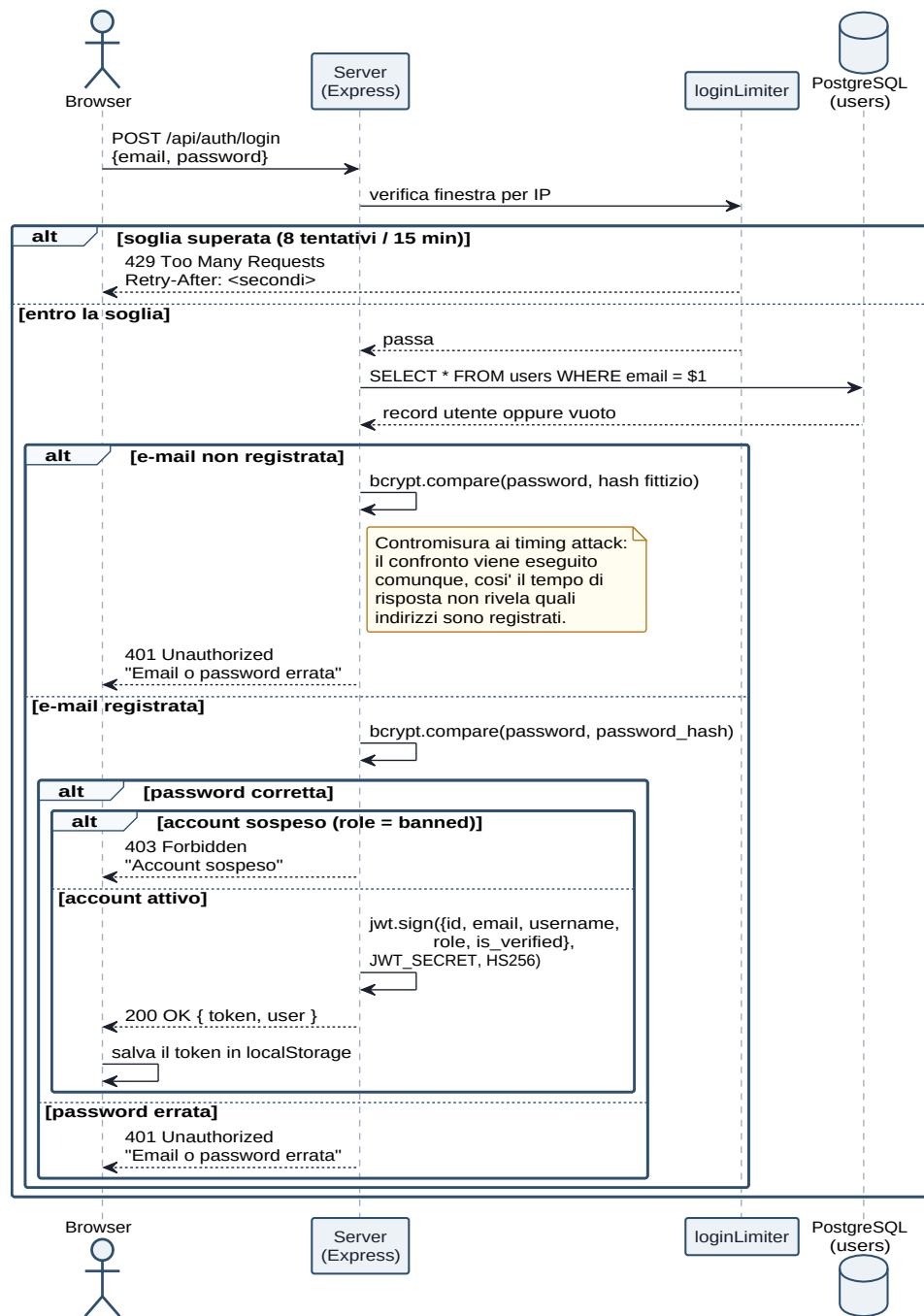


Figura 4.4: Flusso di autenticazione dell'utente.

di amministrazione tramite il middleware `isAdmin`, ed esegue un aggiornamento sulla tabella `users` che imposta il flag `is_verified` a `true`, registra l'istante in `verified_at` e l'identificativo dell'amministratore in `verified_by`. Infine, il server inserisce una nuova riga nella tabella `notifications` con tipo `verified_granted` e come destinatario l'utente appena verificato. Al successivo accesso dell'utente alla piattaforma, sarà presente una nuova notifica nell'interfaccia della piattaforma.

4.4.3 Segnalazione di un contenuto

Un contenuto può essere segnalato nella piattaforma mediante l'apposito bottone presente nella pagina di dettaglio del contenuto. Il flusso descritto è rappresentato nel sequence diagram della Figura 4.6.

Il browser mostra all'utente un form con un menu a tendina contenente le possibili motivazioni della segnalazione, e un campo di testo libero opzionale. All'invio, il browser effettua una richiesta POST all'endpoint `/api/user/videos/:id/report` con il payload della segnalazione. Il server, previa verifica del token, inserisce la segnalazione nella tabella `reports`, con lo stato iniziale di `pending`, associando la segnalazione al video segnalato, all'utente segnalante e alle informazioni fornite. Al termine dell'inserimento, il server restituisce al browser una risposta di conferma.

L'elenco delle segnalazioni viene visualizzato dall'admin tramite il pannello dedicato, tramite una chiamata all'endpoint `GET /api/admin/reports`. Selezionata una segnalazione, l'admin decide se approvarla o rifiutarla. In caso di approvazione il server aggiorna la tabella `reports` impostando lo stato ad `approved` e valorizzando `reviewed_at`, imposta il flag `is_flagged` del video corrispondente e invia all'utente segnalante una notifica di tipo `report_resolved`. In caso di rifiuto, lo stato della segnalazione diventa `rejected` e il video non viene bloccato.

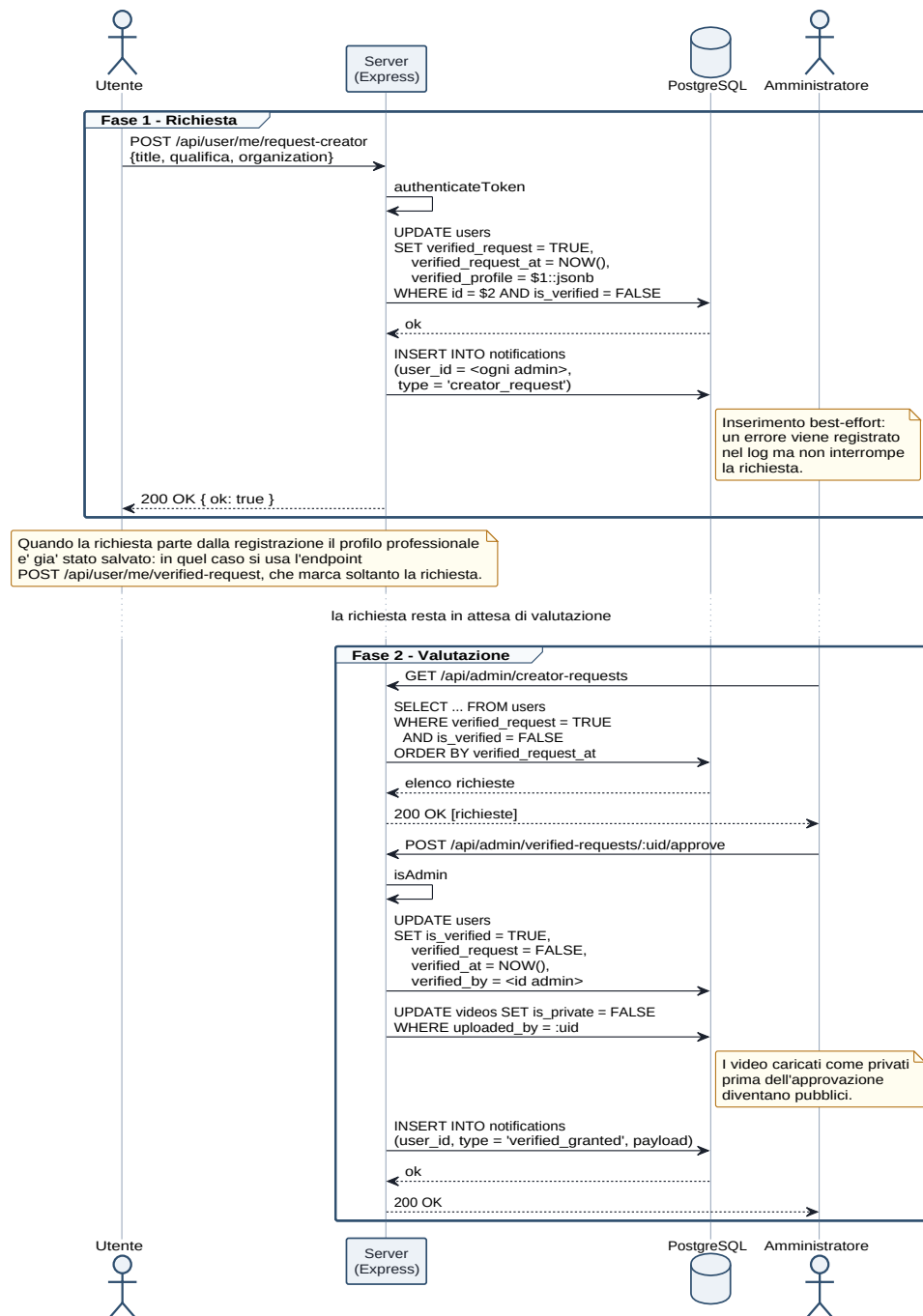


Figura 4.5: Richiesta e approvazione della verifica del profilo creator.

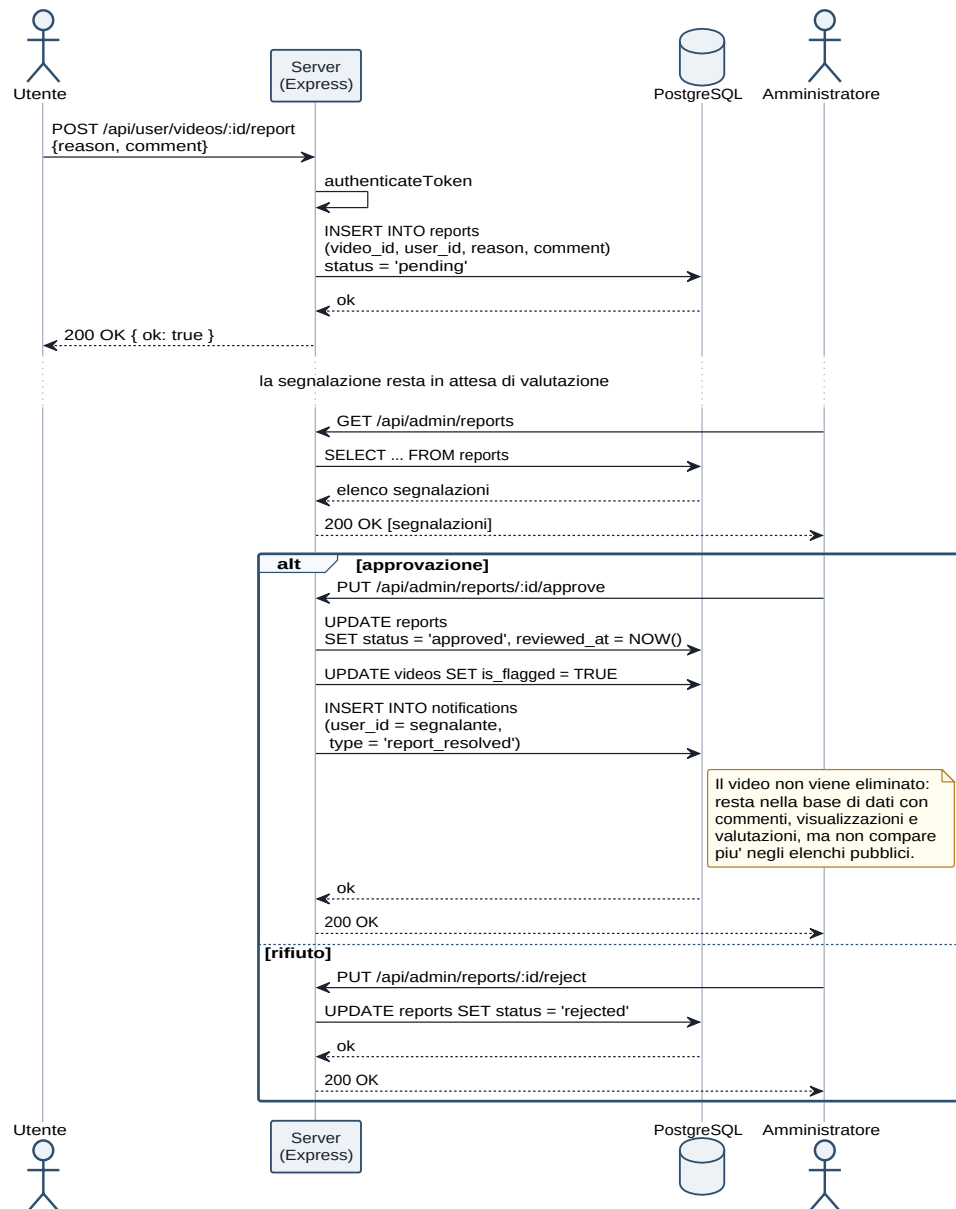


Figura 4.6: Segnalazione di un contenuto e valutazione da parte dell'amministratore.

4.4.4 Moderazione dei commenti e strike

La moderazione di un commento si svolge in quattro passaggi. Un utente segnala un commento e il server inserisce una riga in `comment_reports` con stato `open`. Il creator del video oppure l'amministratore valuta la segnalazione e conferma la rimozione del commento. Dopo questo passaggio il contatore `strike_count` dell'autore del commento viene incrementato. Al terzo strike il ruolo dell'utente passa a `banned` e viene inviata una notifica di tipo `account_banned`. Lo sblocco dell'account richiede un intervento dell'amministratore.

La rimozione dei commenti sotto a un video è affidata al creator che lo ha pubblicato, mentre l'amministratore può intervenire in qualsiasi momento. In questo modo è possibile garantire sostenibilità su un catalogo in crescita, senza concentrare l'intero peso sull'amministratore. Questa scelta comporta però un rischio: un creator può rimuovere i commenti che contestano nel merito i contenuti, e l'autore di quei commenti accumula strike.

4.5 Struttura del backend e progettazione dell'API REST

Il backend di NETMED è stato organizzato utilizzando il principio della separazione delle responsabilità. L'architettura è suddivisa in tre livelli principali: i moduli di routing che gestiscono le richieste HTTP, i middleware che applicano i controlli alle richieste, e i moduli dedicati all'accesso ai dati e ai servizi condivisi. Il file `server.js` costituisce il punto di accesso dell'applicazione e si occupa di configurare Express, registrare i middleware, montare i moduli di routing e avviare il server sulla porta configurata.

L'accesso alle funzionalità del backend viene mediato da un'interfaccia RESTful [39] che espone le risorse principali della piattaforma sotto il prefisso `/api`. La progettazione dell'interfaccia segue le convenzioni dello stile REST: ciascuna risorsa viene identificata da un URL gerarchico e l'operazione viene effettuata tramite il metodo HTTP [40]. Il metodo GET viene utilizzato per la visualizzazione, POST per la creazione di nuovi elementi, PUT per la modifica delle risorse e DELETE per la cancellazione. La gerarchia degli URL rispecchia la relazione di appartenenza tra le risorse. Per esempio, i commenti di un video vengono specificati attraverso il percorso `/api/user/videos/:id/comments` dove l'id è l'identificativo numerico del video. In modo simile le operazioni che riguardano il creator sono raggruppate secondo `/api/creator`.

Le risposte del server utilizzano il formato JSON e adottano una gerarchia di codici di stato HTTP che vengono mostrati nella Tabella 4.1.

Tabella 4.1: Codici di stato HTTP utilizzati nella piattaforma.

Codice	Nome	Significato applicativo
200	OK	Lettura andata a buon fine
201	Created	Nuova risorsa creata (video, commento)
400	Bad Request	Input errato o campo obbligatorio mancante
401	Unauthorized	Token JWT assente
403	Forbidden	Token non valido o scaduto oppure ruolo insufficiente
404	Not Found	Risorsa richiesta inesistente nel database
409	Conflict	Video duplicato
429	Too Many Requests	Superata la soglia di rate limit sull'endpoint
500	Internal Server Error	Eccezione non gestita, tracciata nel log applicativo

4.5.1 Organizzazione dei moduli

La separazione delle responsabilità è presente anche nella struttura delle cartelle del backend, in cui i file sono raggruppati per responsabilità. La struttura risultante è la seguente:

```

NetflixTeleMedicina/
|-- server.js           entry point dell'applicazione
|-- config/
|   '-- security.js     gestione centralizzata di JWT_SECRET
|-- middleware/
|   |-- authMiddleware.js  verifica del token JWT
|   |-- creatorMiddleware.js  verifica dello status di creator
|   '-- rateLimit.js      limitazione sulle rotte critiche
|-- routes/
|   |-- auth.js           endpoint di autenticazione
|   |-- userRoutes.js     API per l'utente comune
|   |-- creatorRoutes.js  API riservate ai creator verificati
|   '-- adminRoutes.js    API riservate agli amministratori
|-- services/
|   |-- passwordPolicy.js  validazione della robustezza password
|   '-- email.js          invio delle e-mail di conferma
|-- db/
|   |-- db.js             pool di connessione a PostgreSQL
|   '-- notifications.js  helper per la creazione delle notifiche
'-- db-init/
    '-- 01_schema.sql     schema SQL eseguito al primo avvio

```

4.5.2 Middleware

I middleware sono meccanismi tramite i quali Express permette di applicare logiche a più rotte, evitando la duplicazione del codice. Nella piattaforma vengono utilizzati diversi middleware, ognuno dei quali garantisce la gestione di un aspetto specifico del controllo delle richieste in ingresso.

Il middleware `authenticateToken`, definito all'interno della cartella `middleware/`, viene eseguito su tutte le rotte che richiedono un utente autenticato. Il suo compito è quello di estrarre il token JWT dall'header della richiesta, verificare la firma utilizzando la chiave segreta che si trova nel modulo `config/security.js` e memorizzare le informazioni dell'utente nel campo `req.user`, rendendole disponibili al gestore delle rotte. In caso di token mancante il middleware restituisce un codice HTTP 401 Unauthorized; se il token è presente ma non valido o scaduto restituisce un 403 Forbidden, così da distinguere le due situazioni.

Il middleware `requireCreator`, definito nella cartella `middleware/`, viene applicato alle rotte riservate ai creator verificati. Si dà per scontato che sulla richiesta sia stato effettuato un `authenticateToken`; il middleware verifica quindi che il flag `is_verified` associato all'utente sia true. In caso contrario restituisce un errore 403.

Il middleware `isAdmin`, definito in `routes/adminRoutes.js` e applicato all'intero router, verifica che l'utente autenticato possieda il ruolo `admin` prima di fornirgli l'accesso alle rotte amministratore.

Il middleware `rateLimit`, sempre situato nella cartella `middleware/`, viene applicato alle rotte di login, di registrazione e di richiesta di reset della password, per evitare attacchi di forza bruta e registrazioni automatizzate. Il numero di tentativi effettuati nella piattaforma provenienti dallo stesso indirizzo IP viene memorizzato in un'apposita struttura, e le richieste in eccesso vengono bloccate restituendo un codice HTTP 429 Too Many Requests.

4.5.3 Rotte suddivise per ruolo

I moduli di routing sono organizzati in quattro file distinti, ognuno dei quali raccoglie gli endpoint destinati a una specifica categoria di utenti così da assicurare una separazione tra le funzionalità accessibili ai ruoli, applicando i middleware richiesti da ciascun modulo.

La Tabella 4.2 riporta gli endpoint principali di ciascun modulo, insieme al metodo HTTP e una breve descrizione del loro funzionamento.

Tabella 4.2: Endpoint REST principali del backend.

Modulo	Metodo	Endpoint	Descrizione
auth.js	POST	/api/auth/register	Registrazione di un nuovo utente
auth.js	POST	/api/auth/login	Autenticazione e rilascio del token JWT
userRoutes.js	GET	/api/user/explore	Elenco dei video pubblici
userRoutes.js	GET	/api/user/videos/:id	Dettaglio di un singolo video
userRoutes.js	POST	/api/user/videos/:id/like	Espressione di gradimento su un video
userRoutes.js	POST	/api/user/videos/:id/report	Segnalazione di un video
userRoutes.js	POST	/api/user/videos/:id/comments	Pubblicazione di un commento
userRoutes.js	POST	/api/user/videos/:id/favorite	Aggiunta o rimozione di un video dai preferiti
userRoutes.js	POST	/api/user/creators/:username/follow	Iscrizione al canale di un creator
userRoutes.js	GET	/api/user/me/notifications	Elenco delle notifiche personali
userRoutes.js	POST	/api/user/me/request-creator	Invio della richiesta di verifica creator
creatorRoutes.js	POST	/api/creator/videos	Pubblicazione di un nuovo video
creatorRoutes.js	PUT	/api/creator/videos/:id	Modifica di un video esistente
creatorRoutes.js	DELETE	/api/creator/videos/:id	Rimozione di un video pubblicato
creatorRoutes.js	DELETE	/api/creator/videos/:id/comments/:cid	Rimozione di un commento sotto un proprio video
adminRoutes.js	GET	/api/admin/users	Elenco degli utenti registrati
adminRoutes.js	PUT	/api/admin/users/:id/role	Modifica del ruolo di un utente
adminRoutes.js	GET	/api/admin/creator-requests	Elenco delle richieste di verifica in attesa
adminRoutes.js	POST	/api/admin/verified-requests/:uid/approve	Approvazione della verifica di un creator
adminRoutes.js	POST	/api/admin/verified-requests/:uid/reject	Rifiuto della verifica di un creator
adminRoutes.js	GET	/api/admin/reports	Elenco delle segnalazioni ricevute
adminRoutes.js	PUT	/api/admin/reports/:id/approve	Approvazione di una segnalazione e blocco del video
adminRoutes.js	PUT	/api/admin/reports/:id/reject	Rifiuto di una segnalazione
adminRoutes.js	GET	/api/admin/audit	Consultazione dell'audit log

L'organizzazione per ruoli permette di applicare il middleware una sola volta, così che tutti gli endpoint ereditino il controllo di ruolo, senza dover attuare la stessa logica per ciascun gestore.

4.5.4 Servizi condivisi e accesso ai dati

Le funzionalità condivise tra le varie rotte sono situate nella cartella `services/`. Lo script `passwordPolicy.js` implementa la logica di validazione della password, tenendo conto dei vari fattori necessari per ottenere una password sicura e rifiutando le password più comuni tramite una lista nera. Il modulo `email.js` si occupa della logica di invio delle e-mail di conferma. Questi moduli vengono invocati direttamente dai gestori delle rotte.

L'accesso al database avviene tramite `db/db.js`, che espone un pool di connessioni PostgreSQL, configurato a partire dalle variabili d'ambiente presenti nel file `.env`. Il pool viene inizializzato al bootstrap dell'applicazione, secondo il funzionamento descritto nella Sezione 4.1.4.

4.5.5 Limitazione del rate di richieste

Per contenere i tentativi di forza bruta e la creazione eccessiva di account, le rotte più esposte sono protette da un meccanismo di rate limiting a finestra scorrevole, implementato senza l'utilizzo di librerie esterne. Il modulo `middleware/rateLimit.js` definisce tre limitatori distinti, applicati alle rotte di login, di registrazione e di richiesta di reset della password.

Il limite è impostato a otto richieste per indirizzo IP nell'arco di quindici minuti sulla rotta di autenticazione e a cinque richieste all'ora sulle rotte di registrazione e di recupero password. Oltre la soglia il server risponde con il codice HTTP 429 (Too Many Requests). Il rate limiting completa le misure relative alla robustezza della password, alla gestione dei token e alla validazione dei payload.

4.6 Struttura del frontend

Il frontend di NETMED è costituito da un insieme di pagine HTML statiche, ciascuna delle quali carica file CSS e i propri script JS. La scelta di adottare il modello Multi-Page Application con JavaScript, descritta e motivata nella Sezione 3.3, si riflette sull'organizzazione del codice sorgente: non esiste alcun processo di build e ciascuna pagina è autonoma dal punto di vista del rendering, con un proprio insieme di risorse.

4.6.1 Organizzazione delle pagine e delle risorse

Tutte le pagine dedicate al frontend sono contenute nella cartella `public/` del progetto, servita da Express come contenuto statico. La struttura della cartella è la seguente.

```
public/
|-- *.html           pagine HTML della piattaforma
```

```
|-- css/
|   |-- user/           fogli di stile delle pagine utente
|   |-- admin/          fogli di stile del pannello admin
|   '-- responsive.css  regole responsive comuni
|-- js/
|   |-- user/           script delle pagine utente
|   |-- admin/          script del pannello admin
|   |-- i18n.js          gestione della multilingua IT/EN
|   '-- cookie-banner.js gestione del banner cookie
|-- img/                immagini statiche (logo, favicon, hero)
'-- sw.js                service worker per le notifiche push
```

Le risorse dedicate agli utenti e amministratori sono mantenute separate sia a livello di fogli di stile sia a livello di script JavaScript. Questa organizzazione consente di evitare sovrapposizioni tra le logiche delle due aree e limita il caricamento di risorse non necessarie.

4.6.2 Interfaccia utente

L'interfaccia utente è composta da ventisei file HTML. Una decina compone il percorso di navigazione principale; le restanti sono pagine di servizio.

La home (Figura 4.7) è la pagina di accesso alla piattaforma e mostra all'utente autenticato una hero image con i video in evidenza, seguita da diverse strisce orizzontali (rows) che raccolgono per categoria i video presenti nella piattaforma. L'organizzazione per la creazione della pagina si è ispirata alle principali piattaforme di streaming offerte dal mercato.

La registrazione (Figura 4.8) è il punto in cui si innesta il meccanismo di verifica: qui l'utente sceglie con quale ruolo iscriversi alla piattaforma, creator o utente ordinario. Se seleziona il ruolo di creator, compare un ulteriore form che richiede la sua specializzazione.

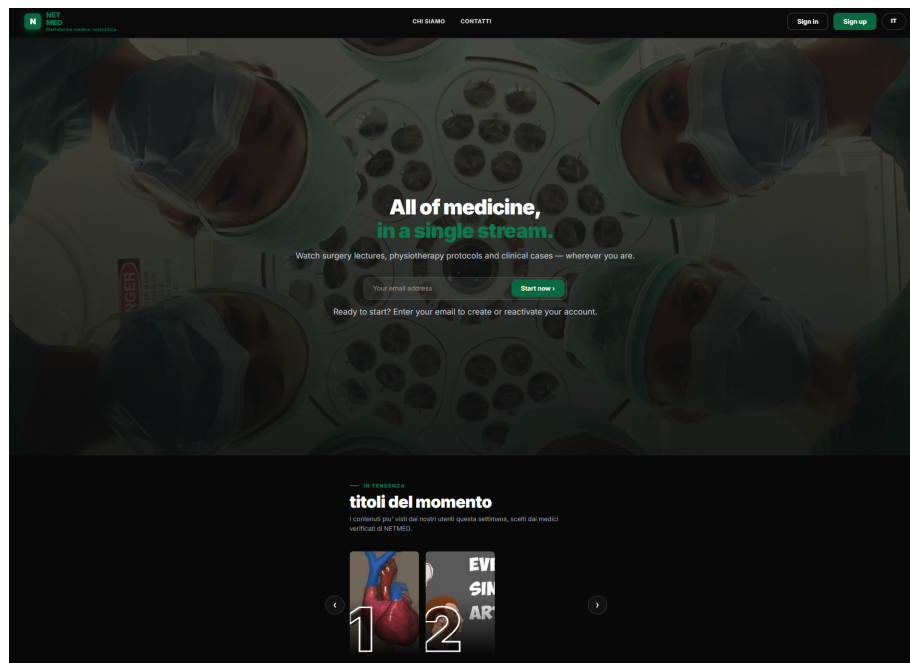


Figura 4.7: Home page di NETMED: in alto la hero image, in basso la sezione con i contenuti in evidenza.

Figura 4.8: Pagina di registrazione, con il modulo aggiuntivo per la richiesta di verifica.

Nella pagina di un video (Figura 4.9) trovano posto il player YouTube incorporato, il titolo, la descrizione, i contatori dei like e delle visualizzazioni, il riferimento al creator con il tesserino di verifica e i commenti.

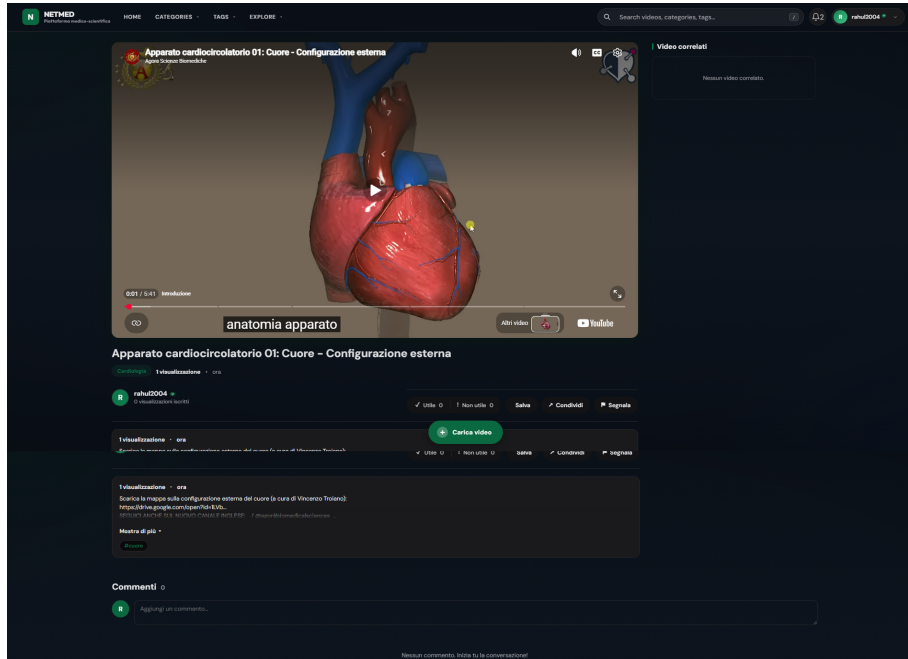


Figura 4.9: Pagina di dettaglio di un video: in alto il player, il titolo, il riferimento al creator con il tesserino di verifica e i pulsanti di interazione; in basso la descrizione, i tag e la sezione dei commenti.

La pagina di ricerca (Figura 4.10) mostra i risultati filtrati per una query, per categoria o per tag. Dal profilo l'utente consulta e modifica le proprie informazioni personali, vede le iscrizioni ai creator, gestisce le cartelle dei contenuti preferiti e ripercorre la cronologia di visualizzazione.

Ai soli creator l'interfaccia aggiunge una pagina, quella di caricamento di un nuovo video (Figura 4.11), dove si inseriscono un identificativo YouTube, un titolo, una descrizione, una categoria e un insieme di tag.

4.6.3 Pannello di amministrazione

Il pannello di amministrazione (Figura 4.12) è realizzato come Single Page Application e raccoglie in un unico strumento operativo le sezioni su cui lavora l'admin: utenti, video, segnalazioni e altre. Il passaggio da una sezione all'altra non richiede il ricaricamento della pagina. Un router minimale aggiorna il contenuto della sezione principale quando l'admin seleziona una voce nella sidebar.

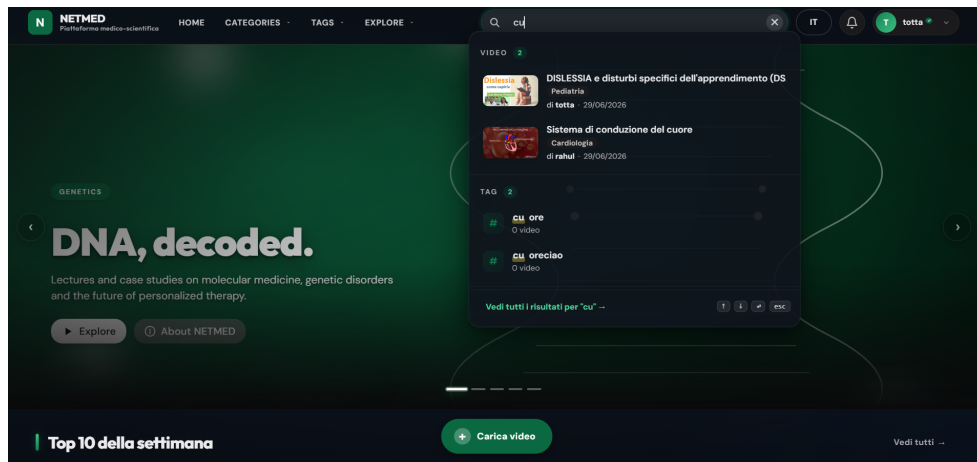


Figura 4.10: Ricerca: i risultati compaiono in un riquadro a comparsa, suddivisi per video e per tag.

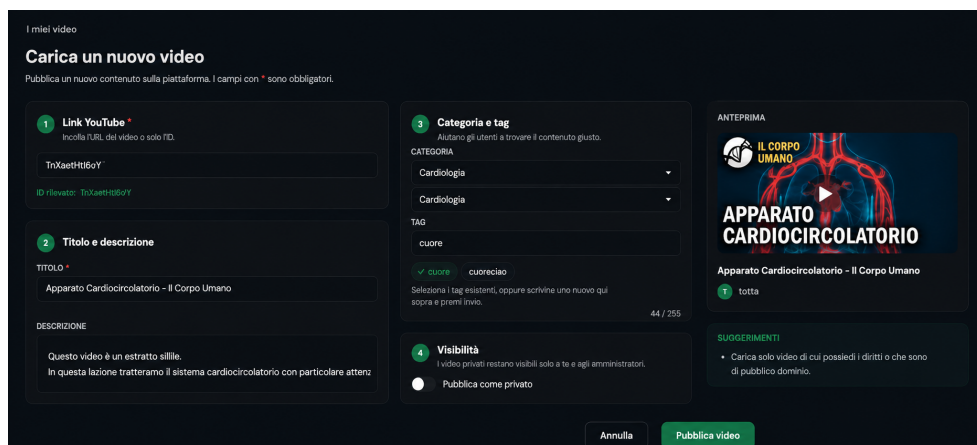


Figura 4.11: Pagina di caricamento di un nuovo video.

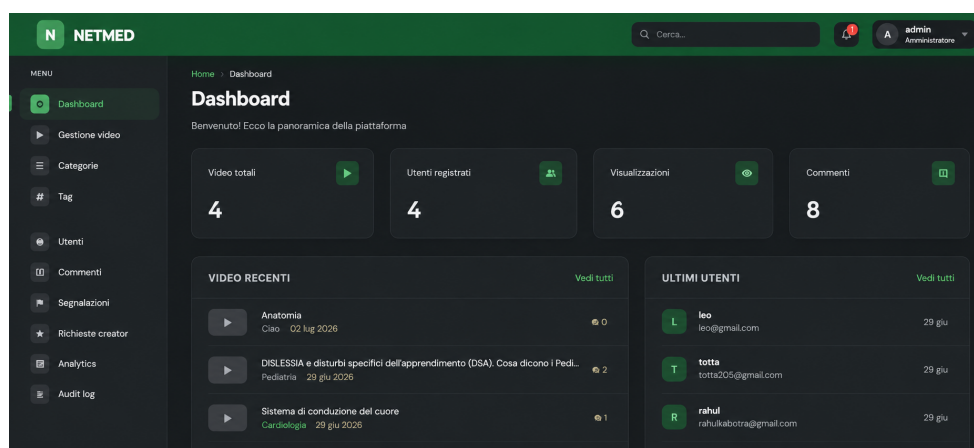


Figura 4.12: Pannello di amministrazione: la sidebar seleziona la sezione, il contenuto centrale viene aggiornato senza ricaricare la pagina.

4.6.4 Componenti trasversali

Alcuni componenti dell'interfaccia sono condivisi da più pagine e sono raccolti in file JavaScript comuni. Il modulo `nm-icons.js` raccoglie le icone SVG della piattaforma. Al passaggio del mouse su un video, `preview-modal.js` ne mostra l'anteprima. Il modulo `feedback-ui.js` offre le notifiche a schermo, dette toast, che informano l'utente sull'esito delle operazioni. La logica della barra di ricerca nell'header sta in `search-v2.js`. Il file `i18n.js` gestisce la multilingua: questa versione di NETMED offre due lingue, italiano e inglese. La preferenza linguistica, che l'utente imposta con l'apposito bottone, resta nel `localStorage` del browser. Lo script `core.js`, incluso in tutte le pagine, contiene le chiamate API con autenticazione automatica e gestisce il tema visivo, che l'utente può scegliere tra chiaro e scuro. Anche questa preferenza è applicata all'apertura di ogni pagina, così l'aspetto resta coerente su tutta la piattaforma.

4.7 Sistema di verifica dei creator

Il sistema di verifica dei creator è una delle principali funzionalità di NETMED, e ha l'obiettivo di offrire agli utenti contenuti pubblicati da autori la cui qualifica è stata controllata dall'admin. Il processo si svolge in due fasi distinte, già descritte nella Sezione 4.4.2; questa sezione ne illustra l'implementazione.

4.7.1 Modello dei dati

La verifica non ha richiesto alcuna nuova tabella: i dati restano nella tabella `users`. Il campo principale è il flag booleano `is_verified`, che indica lo stato corrente: false per

un utente ordinario, true per un creator verificato. L'istante in cui l'utente presenta la richiesta finisce nel timestamp `verified_request_at`, così l'amministratore può ordinare le richieste cronologicamente. Alla data di accettazione è dedicato `verified_at`, mentre `verified_by` conserva l'id dell'amministratore che ha approvato, per tenere traccia dell'operazione. Il campo `verified_profile`, di tipo JSONB, contiene le informazioni professionali dichiarate dall'utente.

4.7.2 Fase di richiesta

L'utente presenta la richiesta di verifica al momento della registrazione, oppure più tardi dal proprio profilo. Il form richiede le informazioni sul titolo personale, la specializzazione clinica o accademica e l'organizzazione di appartenenza. Per semplificare l'inserimento dei dati il form utilizza elementi di tipo `datalist`, dai quali l'utente può selezionare una voce esistente o inserirne una nuova. L'endpoint `/api/user/me/request-creator` costruisce il JSON del profilo e lo memorizza nel campo `verified_profile`, aggiornando nello stesso momento il campo `verified_request_at`. Al termine dell'operazione l'utente riceve una risposta di conferma. Il ruolo non cambia: in questa fase l'utente non ha ancora accesso alle funzionalità di creator.

4.7.3 Fase di valutazione

Durante la fase di valutazione, l'amministratore visualizza le richieste in arrivo nell'apposita pagina `admin_richieste.html`. L'endpoint `GET /api/admin/creator-requests` le recupera, restituendo le righe della tabella `users` con `verified_request_at` valorizzato e `is_verified` impostato a false. Per ciascuna richiesta l'amministratore visualizza lo username dell'utente, la sua e-mail, il contenuto del JSONB `verified_profile` e la data della richiesta; a quel punto ha due possibilità.

L'approvazione invia una richiesta `POST` all'endpoint `/api/admin/verified-requests/:uid/approve`. Il gestore della rotta aggiorna la tabella `users`: imposta il flag `is_verified` a true, valorizza `verified_at` e memorizza l'identificativo dell'admin che ha approvato. Subito dopo, una notifica di tipo `verified_granted` raggiunge l'utente appena verificato. Il rifiuto funziona in modo analogo, inviando una richiesta all'endpoint `/api/admin/verified-requests/:uid/reject`, che azzerà i campi della richiesta e invia una notifica di tipo `creator_request_denied`. L'utente può ripresentare la richiesta in un secondo momento, dopo aver modificato i campi.

4.7.4 Approvazione e revoca della verifica

Una volta che l'admin ha approvato, il flag `is_verified` apre all'utente tutte le funzionalità riservate al creator: il middleware `requireCreator`, descritto nella Sezione 4.5.2, lo riconosce come autorizzato e lascia passare le richieste dirette alle rotte di

`creatorRoutes.js`. Nell'interfaccia compare il tesserino di verifica. La verifica può essere revocata in un secondo momento, se l'admin ha dei dubbi o se l'utente viola i termini di utilizzo della piattaforma. La revoca passa dall'endpoint `DELETE /api/admin/users/:id/verify`, che riporta il flag `is_verified` a false, registra l'operazione nella tabella `admin_audit` e azzerà tutti i campi `timestamp`. I video non sono eliminati: restano nella base di dati e il campo `is_private` li rende privati.

4.8 Sistema di notifiche in app

Il sistema di notifiche interne alla piattaforma consente al server di comunicare agli utenti determinati eventi che si verificano durante la navigazione, oppure che riguardano il proprio profilo o i propri contenuti.

Le notifiche in app affiancano l'invio di messaggi di posta elettronica, che nella piattaforma resta riservato alle operazioni sull'account. Per gli eventi ordinari si è usata la notifica interna, che non dipende dalla disponibilità di un server SMTP e resta consultabile in qualsiasi momento. Questo rende il deployment più semplice e riduce il numero di configurazioni necessarie. La memorizzazione nella base di dati permette inoltre di tenere traccia di tutte le comunicazioni passate. Le tipologie di notifica previste sono riportate nella Tabella 4.3.

Tabella 4.3: Tipologie di notifiche presenti in NETMED.

Tipo	Trigger	Payload principale
<code>welcome</code>	registrazione	<code>username</code>
<code>creator_request</code>	richiesta di verifica	<code>user_id</code> del richiedente
<code>verified_granted</code>	approvazione admin	<code>admin_id</code> , <code>admin_name</code>
<code>creator_request_denied</code>	rifiuto admin	<code>admin_id</code>
<code>verified_revoked</code>	revoca della verifica	<code>actor_id</code> , <code>videos_hidden</code>
<code>report_resolved</code>	admin approva segnalazione	<code>video_id</code> , <code>title</code> , <code>reason</code>
<code>new_video</code>	upload di un creator seguito	<code>video_id</code> , <code>title</code>

4.8.1 Modello dei dati e tipi di notifica

La tabella `notifications` memorizza tutte le notifiche. Ciascuna riga contiene il riferimento all'utente destinatario, un campo `type` che identifica la tipologia dell'evento, un campo `read_at` che segnala se la notifica è stata letta, la data di creazione e un campo `payload` di tipo JSONB con le informazioni variabili sull'evento. Per `verified_granted` il payload contiene l'id e l'username dell'admin che ha approvato; per `report_resolved`, l'id del video coinvolto, il suo titolo e il motivo della segnalazione.

4.8.2 Creazione secondo il paradigma best-effort

La funzione `createNotification` inserisce la riga nella tabella `notifications`, ed è invocata dai gestori delle rotte dopo il completamento delle operazioni principali. La query sta dentro un blocco `try/catch`, che cattura eventuali errori. L'approccio `best-effort` separa così la gestione delle notifiche dall'operazione principale. Se, per esempio, si verifica un errore nel database o nel contenuto del payload, l'operazione principale a cui la notifica era collegata non è interrotta.

4.8.3 Consumo dal client

Il client legge le notifiche con una chiamata all'endpoint `GET /api/user/me/notifications`, che fornisce l'elenco di quelle dirette all'utente autenticato. La chiamata parte all'apertura di ciascuna pagina, così l'elenco resta aggiornato. Alla selezione dell'icona della campanella il browser mostra una finestra a comparsa con le notifiche più recenti; alla lettura di una notifica l'endpoint `PUT /api/user/me/notifications/:id/read` aggiorna il campo `read_at`, così le nuove restano distinguibili da quelle già visualizzate.

4.9 Implementazione dei componenti più significativi

Per chiarire le componenti presentate nelle sezioni precedenti, questa sezione riporta alcuni estratti del codice sorgente del backend. I frammenti scelti sono quelli più rappresentativi dell'architettura complessiva e delle scelte progettuali adottate. Ogni estratto è seguito da un commento tecnico che ne evidenzia i punti principali.

4.9.1 Middleware di autenticazione

Il middleware `authenticateToken`, contenuto nel file `middleware/authMiddleware.js`, interviene prima che la richiesta raggiunga il gestore della rotta, e presuppone che il client inserisca il token JWT nell'header `HTTP Authorization`. Il Codice 4.1 ne riporta l'implementazione.

```
1  /*
2   Middleware che protegge le rotte che richiedono un utente autenticato.
3   - Legge l'header "Authorization: Bearer <jwt>"
4   - Verifica firma e scadenza con JWT_SECRET
5   - Proietta il payload decodificato in req.user
6   Uso: router.get("/private", authenticateToken, handler)
7  */
8
9  const jwt = require("jsonwebtoken");
10
```

```
11 // JWT_SECRET risolto centralmente da config/security.js:
12 // in produzione muore se non e' settato, in dev usa un fallback noto.
13 const { JWT_SECRET } = require("../config/security");
14
15 function authenticateToken(req, res, next) {
16   const authHeader = req.headers["authorization"];
17   const token = authHeader && authHeader.split(" ")[1]; // "Bearer <token>"
18
19   // 401 = non ti sei presentato (manca del tutto il token).
20   if (!token) return res.status(401).json({ error: "Token mancante" });
21
22   jwt.verify(token, JWT_SECRET, (err, user) => {
23     // 403 = ti sei presentato ma il token e' invalido o scaduto.
24     if (err) return res.status(403).json({ error: "Token non valido" });
25     req.user = user; // { id, email, username }
26     next();
27   });
28 }
29
30 module.exports = authenticateToken;
```

Codice 4.1: Middleware di autenticazione JWT (middleware/authMiddleware.js).

Tre elementi meritano attenzione. Il primo è la chiave segreta `JWT_SECRET`, importata dal modulo `config/security.js`, che impone la definizione della chiave come variabile d'ambiente e interrompe l'avvio dell'applicazione se non la trova. Il secondo è la distinzione dei codici 401 e 403, che rappresentano la differenza tra token mancante e token non valido o scaduto. Il terzo è l'assegnazione del payload decodificato a `req.user`, che mette a disposizione del gestore le informazioni sull'utente senza una nuova interrogazione alla base di dati.

4.9.2 Creazione delle notifiche

Il modulo `db/notifications.js` implementa la creazione delle notifiche secondo il paradigma best-effort come descritto nella Sezione 4.8.2. Il Codice 4.2 mostra l'implementazione della funzione `createNotification`.

```
1
2 const pool = require("../db");
3
4 /*
5   Crea una notifica per un utente.
6   */
7
8 async function createNotification(userId, type, payload, link) {
9   if (!userId || !type) return;
10  try {
```

```

11   await pool.query(
12     'INSERT INTO notifications (user_id, type, payload, link)
13     VALUES ($1, $2, $3::jsonb, $4)',
14     [userId, type, JSON.stringify(payload || {}), link || null]
15   );
16   } catch (e) {
17     console.warn("[notifications] insert failed:", e.code || "", e.message);
18   }
19 }
20
21 /*
22  Variante che evita di notificare l'attore stesso.
23  */
24 async function createNotificationIfDifferent(actorId, recipientId, type, payload, link)
25 {
26   if (!recipientId || actorId === recipientId) return;
27   return createNotification(recipientId, type, payload, link);
28 }
29
30 module.exports = {
31   createNotification,
32   createNotificationIfDifferent,
33 };

```

Codice 4.2: Helper per la creazione delle notifiche (db/notifications.js).

Il blocco try/catch che circonda la query di inserimento è il punto più significativo del codice. Un errore come la violazione di un vincolo o un problema di connessione finisce nel log applicativo tramite `console.warn`, ma non è propagato al chiamante. L'operazione che ha generato la notifica prosegue quindi anche quando la notifica non può essere salvata. La funzione `createNotificationIfDifferent` introduce un controllo per evitare che l'utente riceva notifiche da eventi generati da lui stesso.

4.9.3 Caricamento di un video

Il modulo `routes/creatorRoutes.js` gestisce il caricamento dei video: validazione dell'input, creazione di categoria e tag e infine inserimento nella tabella `videos`. Il Codice 4.3 ne riporta la rotta principale.

```

1 // routes/creatorRoutes.js
2 // =====
3 // POST /api/creator/videos
4 // Carica un nuovo video. Identico al POST admin/videos ma con
5 // uploaded_by = req.user.id (forzato, non leggibile dal body).
6 // =====
7 router.post("/videos", async (req, res) => {
8   try {

```

```
9   const youtube_id = (req.body.youtube_id || "").trim().slice(0, 20);
10  const title = (req.body.title || "").trim().slice(0, 255);
11  const description = (req.body.description || "").trim().slice(0, 4000);
12  let category_id = parseInt(req.body.category_id, 10) || null;
13  const is_private = !!req.body.is_private;
14  const tag_ids = safeTagIds(req.body.tag_ids);
15  const new_tags = safeTagNames(req.body.new_tag_names);
16  const new_cat = (req.body.new_category_name || "").toString().trim().slice(0, 80);
17
18  if (!youtube_id || !title) {
19    return res.status(400).json({ error: "youtube_id e title obbligatori" });
20  }
21  // Validazione veloce dell'ID YouTube (alfanumerico + _ + -, 11 char tipico)
22  if (!/^[a-zA-Z0-9_+ -]{6,20}$/.test(youtube_id)) {
23    return res.status(400).json({ error: "youtube_id non valido" });
24  }
25
26  // Se l'utente ha digitato una nuova categoria, la creo (o riuso).
27  if (!category_id && new_cat) {
28    category_id = await resolveOrCreateCategoryId(new_cat);
29  }
30
31  // Se l'utente ha digitato nuovi nomi tag, li trasformo in id e li
32  // unisco a quelli scelti dal dropdown (dedup).
33  let allTagIds = tag_ids.slice();
34  if (new_tags.length) {
35    const created = await resolveOrCreateTagIds(new_tags);
36    for (const id of created) if (!allTagIds.includes(id)) allTagIds.push(id);
37  }
38
39  const r = await pool.query(
40    'INSERT INTO videos
41      (youtube_id, title, description, thumbnail_url, uploaded_by, category_id,
42       is_private)
43      VALUES ($1, $2, $3, $4, $5, $6, $7)
44      RETURNING *',
45    [
46      youtube_id,
47      title,
48      description || null,
49      ytThumb(youtube_id),
50      req.user.id,
51      category_id,
52      is_private,
53    ]
54  );
55  if (allTagIds.length) {
```

```

56     const vals = allTagIds.map((_, i) => `(${1},${i + 2})`).join(",");
57     await pool.query(
58       'INSERT INTO video_tags(video_id, tag_id) VALUES ${vals} ON CONFLICT DO NOTHING',
59       [r.rows[0].id, ...allTagIds]
60     );
61   }
62
63   // Notifica all'admin: utile per moderazione di secondo livello.
64   try {
65     await pool.query(
66       "INSERT INTO admin_notifications(type,title,message,related_id) VALUES($1,$2,$3,
67         $4)",
68       [
69         "new_video",
70         "Video da utente verificato",
71         `${req.user.username} ha pubblicato "${title}"`,
72         r.rows[0].id,
73       ]
74     );
75   } catch {}
76
77   // Push agli iscritti del creator (best effort, non blocca la response).
78   if (!is_private) {
79     try {
80       const userRoutes = require("./userRoutes");
81       if (typeof userRoutes.notifyFollowersNewVideo === "function") {
82         userRoutes.notifyFollowersNewVideo(req.user.id, r.rows[0]).catch(() => {});
83       }
84     } catch (_) {}
85   }
86
87   res.status(201).json(r.rows[0]);
88 } catch (e) {
89   if (e.code === "23505") {
90     return res.status(409).json({ error: "Questo video YouTube è già in piattaforma" });
91   }
92   console.error(e);
93   res.status(500).json({ error: "Errore interno" });
94 }
95 });

```

Codice 4.3: Caricamento di un video (routes/creatorRoutes.js).

Il campo `uploaded_by` è assegnato lato server con `req.user.id`, estratto dal token JWT, così il client non può dichiarare un autore diverso da sé. Nel caso in cui il video sia già presente, PostgreSQL restituisce il codice 23505, che corrisponde alla violazione del vincolo UNIQUE su `youtube_id`: l'errore viene trasformato in una risposta HTTP 409, evitando così di restituire un generico errore interno. I middleware non compaiono

nella firma della rotta perché sono applicati all'intero router in `server.js`: ogni endpoint di `creatorRoutes.js` attraversa `authenticateToken` e `requireCreator` prima di raggiungere il gestore.

4.9.4 Registrazione utente con validazione e hashing bcrypt

L'endpoint di registrazione riunisce la validazione dell'input e la protezione della password. Il Codice 4.4 ne riporta l'implementazione.

```
1 // routes/auth.js
2 // Endpoint per la registrazione
3 router.post("/register", registerLimiter, async (req, res) => {
4   try {
5     const { email, username, password } = req.body;
6     const wantsCreator = !(req.body && req.body.wants_creator);
7     const cTitle = ((req.body && req.body.creator_title) || "").toString().trim().slice
8       (0, 60);
9     const cQualifica = ((req.body && req.body.creator_qualifica) || "")
10       .toString()
11       .trim()
12       .slice(0, 120);
13     const cOrg = ((req.body && req.body.creator_org) || "").toString().trim().slice(0,
14       160);
15
16     if (!email || !username || !password) {
17       return res.status(400).json({ error: "Campi mancanti" });
18     }
19
20     // Validazione formato email lato server (cintura + bretelle)
21     if (!/^[\s@]+@[^\s@]+\.[^\s@]+$/ .test(email) || email.length > 120) {
22       return res.status(400).json({ error: "Email non valida" });
23     }
24
25     // Username: solo lettere, numeri, underscore/punto, 3-30 char
26     if (!/^[a-zA-Z0-9_]{3,30}$/ .test(username)) {
27       return res.status(400).json({ error: "Username 3-30 caratteri, solo lettere/numeri/
28         _/." });
29     }
30
31     // Password policy unica
32     const pCheck = validatePassword(password);
33     if (!pCheck.ok) return res.status(400).json({ error: pCheck.error });
34
35     // check email
36     const emailCheck = await pool.query("SELECT id FROM users WHERE email = $1", [email
37       ]);
38     if (emailCheck.rows.length > 0) {
39       return res.status(400).json({ error: "Email già registrata" });
40     }
41
42     // check username
```

```
36     const usernameCheck = await pool.query("SELECT id FROM users WHERE username = $1", [
37         username]);
38     if (usernameCheck.rows.length > 0) {
39         return res.status(400).json({ error: "Username già registrato" });
40     }
41     // Hash password (bcrypt 10 round)
42     const hash = await bcrypt.hash(password, 10);
43     // Inserimento nel DB
44     const insertResult = await pool.query(
45         'INSERT INTO users (email, username, password_hash)
46         VALUES ($1, $2, $3)
47         RETURNING id, email, username',
48         [email, username, hash]
49     );
50     const newUserId = insertResult.rows[0].id;
51
52     // [omesse: notifica di benvenuto, invio dell'e-mail di conferma
53     // e registrazione dell'eventuale richiesta di verifica creator]
54
55     res.status(201).json({
56         message: "Utente registrato con successo",
57         user: insertResult.rows[0],
58     });
59 } catch (err) {
60     console.error("ERRORE DURANTE REGISTRAZIONE:", err);
61     res.status(500).json({ error: err.message });
62 }
63 });
```

Codice 4.4: Registrazione utente con validazione e hashing (`routes/auth.js`).

La rotta è protetta dal middleware `registerLimiter`, che limita il numero di richieste per indirizzo IP. La validazione dell'input avviene interamente lato server: il codice controlla il formato dell'indirizzo e-mail e vincola lo username a un insieme di caratteri ammessi. La funzione `validatePassword` impone una password robusta, con una lunghezza minima e la presenza di categorie di caratteri diverse. La password non è mai salvata in chiaro: `bcrypt` la converte in un hash con fattore di costo pari a dieci. L'hash è memorizzato nella colonna `password_hash` della tabella `users`. L'uso di `bcrypt` aumenta il costo computazionale dei tentativi di brute force rispetto a funzioni di hash generiche.

5

Valutazione del sistema

Il sistema è stato valutato seguendo due criteri. Il primo è la correttezza funzionale del backend, verificata tramite una suite di test automatici sugli endpoint dell'API e le politiche di autorizzazione descritte nel Capitolo 4. Il secondo sono le prestazioni degli endpoint principali sotto carico. Il capitolo riporta le misurazioni ottenute, l'ambiente di lavoro e le scelte progettuali.

5.1 Obiettivi della valutazione

Il primo obiettivo della valutazione è verificare che l'implementazione rispetti le politiche di autorizzazione descritte nel Capitolo 4. Le regole di accesso sono un punto fondamentale, perché distinguono i vari utenti della piattaforma, e un singolo errore può compromettere la garanzia professionale che si vuole offrire. La suite contiene, per ciascuno dei tre ruoli, casi che verificano il rifiuto delle richieste prive dei permessi necessari. Il secondo obiettivo è quello di stabilire se i tempi di risposta sono compatibili con un uso interattivo.

5.2 Ambiente e metodologia di test

La suite è scritta con Mocha [41] per l'esecuzione, Chai [42] per le asserzioni e Supertest [43] per l'invio delle richieste. Supertest costruisce le richieste direttamente sull'istanza

Express esportata da `server.js`, senza aprire porte di rete: quando `NODE_ENV` vale `test` il server non invoca `listen`.

La stessa variabile disattiva il rate limiter descritto nella Sezione 4.5.5. L'eccezione è necessaria: la suite invia centinaia di richieste in pochi secondi e senza di essa verrebbe bloccata dal codice 429.

Le misure riportate nel capitolo provengono da un portatile con processore Intel Core i7-1255U e sistema operativo Windows 11. L'applicazione è eseguita su Docker Desktop [35] e utilizza i container descritti nella Sezione 4.1.3: `netmed-app` con Node.js 18 e `netmed-db` con PostgreSQL 16 [30]. I valori dipendono dall'hardware; restano invece invariati i rapporti fra le misure e i piani di esecuzione scelti da PostgreSQL.

5.2.1 Copertura delle funzionalità principali

La suite è composta da tre file, uno per ogni ruolo.

Con ventisei casi di test, `test-utente-api.js` copre la navigazione pubblica e le operazioni dell'utente registrato: l'elenco dei video, la ricerca per testo o categoria, il dettaglio di un contenuto, i commenti in scrittura e in lettura, i preferiti, la cronologia di visualizzazione e le notifiche. Il file `test-creator-api.js`, con dieci casi di test, copre la gestione dei video, cioè la pubblicazione, la modifica e la rimozione di un contenuto, insieme alla moderazione dei commenti sotto i propri video. Il file `test-admin-api.js`, con ventidue casi di test, esercita il pannello di amministrazione, quindi tutto ciò che riguarda la gestione degli utenti, dei ruoli, delle richieste di verifica e delle segnalazioni.

Ogni file prepara i dati e li rimuove al termine delle operazioni; in questo modo le suite restano indipendenti e sono eseguibili in qualsiasi ordine. L'utente di prova viene creato direttamente nella base di dati, con l'hash bcrypt già calcolato e un token JWT firmato, senza passare dall'endpoint di login. In questo modo, in caso di guasto del flusso di autenticazione, falliscono solo i casi che lo riguardano, e non tutti gli altri.

5.2.2 Verifica delle politiche di autorizzazione

Su un campione di endpoint riservati sono presenti casi che verificano il rifiuto delle richieste prive dei permessi necessari. Tre esempi per chiarire la tecnica utilizzata. Una richiesta a `GET /api/user/me` priva del token deve ricevere sempre un codice 401. La stessa richiesta diretta a `GET /api/admin/stats` da parte di un utente ordinario deve ricevere 401 o 403. Una richiesta `GET /api/creator/my/videos` inviata da un utente non verificato deve ricevere 403 anche quando il token è valido, perché il middleware `requireCreator` non si accontenta di un utente autenticato, ma richiede il flag `is_verified`.

Il valore di questi casi non sta nel confermare che il middleware funzioni: questa verifica è possibile anche manualmente. Sta nell'intercettare future regressioni: se una

rotta venisse spostata da un modulo di routing a un altro, o un middleware venisse cancellato durante una modifica, il caso negativo fallirebbe immediatamente e segnalerebbe il problema prima che raggiunga l'utenza.

5.2.3 Esito della suite

La suite conta 58 casi, ripartiti come riportato nella Tabella 5.1.

Tabella 5.1: Ripartizione dei casi di test della suite automatica.

File	Ambito	Casi
<code>test-utente-api.js</code>	Navigazione, commenti, profilo, notifiche	26
<code>test-creator-api.js</code>	Pubblicazione e moderazione dei contenuti	10
<code>test-admin-api.js</code>	Amministrazione e moderazione	22
Totale		58

L'esecuzione termina in circa due secondi e tutti i 58 casi di test passano. Il numero non è una misura di qualità, poiché un test può essere superficiale; conta la distribuzione, in particolare il fatto che i casi negativi sull'autorizzazione sono presenti per tutti e tre i ruoli.

5.3 Tempi di risposta degli endpoint principali

5.3.1 Metodologia di misurazione

Le misurazioni sono state raccolte con autocannon [44], uno strumento di load test per applicazioni Node.js. Ogni endpoint è stato sollecitato per dieci secondi con dieci connessioni concorrenti; lo strumento raccoglie il tempo di risposta medio, il novantacinquesimo e il novantanovesimo percentile e il numero di richieste servite al secondo.

Per questa misurazione, il rate limiter è stato disattivato tramite la variabile d'ambiente `RATE_LIMIT_OFF`. Sulla rotta di autenticazione il limite è di otto tentativi per quindici minuti: con dieci connessioni concorrenti il limite scatta in meno di un secondo e tutte le successive richieste sarebbero 429, rendendo di fatto la misura priva di significato. L'oggetto della misura è il tempo di risposta dell'endpoint, non l'efficacia del limitatore. Le misurazioni sono avvenute su un catalogo di 50.002 video, di cui 50.000 generati tramite uno script.

5.3.2 Risultati

La Tabella 5.2 riporta i valori raccolti.

Tabella 5.2: Tempi di risposta degli endpoint principali, misurati con dieci connessioni concorrenti per dieci secondi su un catalogo di 50.002 video.

Endpoint	Metodo	Media	p95	p99	Ric./s
/api/user/videos/:id/comments	GET	5,2 ms	8,4 ms	12,2 ms	1928
/api/user/explore	GET	8,6 ms	12,6 ms	15,7 ms	1160
/api/user/videos/:id	GET	10,8 ms	14,5 ms	17,8 ms	927
/api/user/me/notifications	GET	14,8 ms	23,6 ms	30,3 ms	675
/api/admin/reports	GET	23,7 ms	33,2 ms	39,7 ms	422
/api/user/videos/:id/like	POST	25,4 ms	35,8 ms	41,0 ms	393
/api/auth/login	POST	53,9 ms	58,3 ms	61,9 ms	56

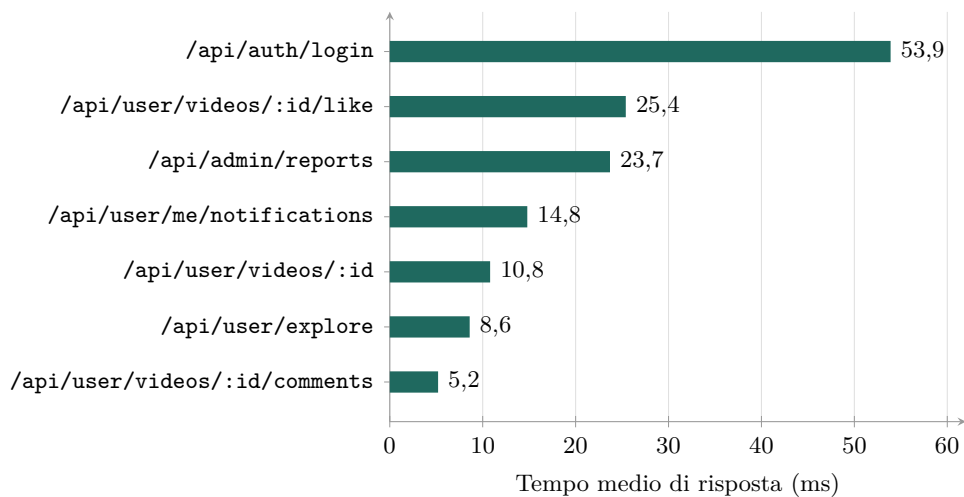


Figura 5.1: Tempo medio di risposta degli endpoint principali, ordinati per valore crescente. L'endpoint di autenticazione è di un ordine di grandezza più lento degli altri.

Tutte le richieste inviate durante le prove hanno ottenuto codice 200; non si sono verificati errori né timeout. I valori provengono da una singola esecuzione per endpoint: sono indicativi dell'ordine di grandezza e non sono corredati da una stima della variabilità.

5.3.3 Discussione dei risultati

Gli endpoint di lettura hanno una media sotto i venticinque millisecondi. Il feed pubblico serve più di mille richieste al secondo pur interrogando una tabella da cinquantamila righe: il merito è dell'indice discusso nella Sezione 5.4.2: senza di esso le interrogazioni avrebbero costi maggiori.

L'endpoint `/api/auth/login` risponde in 53,9 ms, circa dieci volte il tempo dell'endpoint più veloce. La causa è `bcrypt` [24]: misurando la sola funzione di hash sullo stesso hardware, con cost factor pari a dieci, si ottiene un tempo medio di 43,9 ms per hash, cioè l'ottantuno per cento del tempo di risposta dell'endpoint. Il costo è voluto e rende impraticabile la ricerca esaustiva dello spazio delle password, secondo il ragionamento fatto nella Sezione 3.1.4. Quaranta millisecondi per un'operazione che l'utente compie una volta per sessione sono un prezzo ragionevole; lo stesso costo su un endpoint di lettura interrogato a ogni apertura di pagina sarebbe inaccettabile. Il cost factor va comunque rivalutato nel tempo: al crescere della potenza di calcolo il valore andrà alzato.

5.4 Ottimizzazione dell'accesso ai dati

5.4.1 Indici sulle colonne di ricerca

Lo schema del database definisce indici sulle colonne usate come chiavi esterne e su quelle che compaiono nelle interrogazioni più frequenti. Alcuni indici sono parziali, definiti con una clausola `WHERE` che limita l'indicizzazione alle sole righe che soddisfano la condizione. Sulla tabella `videos`, l'unico indice presente nello schema iniziale era `idx_videos_category`, sulla colonna `category_id`. L'interrogazione del feed pubblico non filtra per categoria: ordina per `created_at`, e su quella colonna non era presente alcun indice.

5.4.2 Effetto dell'indice sul feed pubblico

L'interrogazione in esame è quella sull'endpoint `GET /api/user/explore`, nel Codice 5.1. Seleziona l'identificativo, il titolo, l'anteprima e la data di ciascun video, la categoria e l'autore tramite due join esterni e calcola i conteggi delle visualizzazioni e dei voti positivi tramite due sottoquery. L'ordinamento è per data, e il numero di righe restituite è limitato a quarantotto.

```
1 SELECT v.id, v.youtube_id, v.title, v.thumbnail_url, v.created_at,
2       c.name AS category_name, c.id AS category_id,
3       u.username AS uploaded_by_username,
4       u.is_verified AS uploaded_by_verified,
5       (SELECT COUNT(*) FROM views WHERE video_id = v.id)::int AS views_count,
6       (SELECT COUNT(*) FROM likes WHERE video_id = v.id AND vote = 1)::int
7       AS useful_count
8 FROM videos v
9 LEFT JOIN categories c ON c.id = v.category_id
10 LEFT JOIN users u ON u.id = v.uploaded_by
11 WHERE v.is_private = FALSE AND v.is_flagged = FALSE
12 ORDER BY v.created_at DESC
```

13 `LIMIT $1;`

Codice 5.1: Interrogazione del feed pubblico (`routes/userRoutes.js`).

La misurazione è avvenuta in quattro configurazioni, con lo script `scripts/tesi-esperimento-indice.sql`. Per ciascuna è stato raccolto il piano di esecuzione tramite `EXPLAIN (ANALYZE, BUFFERS)`. I risultati sono riportati nella Tabella 5.3.

Tabella 5.3: Costo dell'interrogazione del feed pubblico nelle quattro configurazioni misurate.

Configurazione	Piano di esecuzione	Tempo	Dim. indice
2 video, senza indice	Seq Scan + quicksort	0,53 ms	—
50.002 video, senza indice	Seq Scan completo + top-N heap-sort su 46.062 righe	22,98 ms	—
50.002 video, indice completo	Index Scan con filtro	0,42 ms	1112 kB
50.002 video, indice parziale	Index Scan senza filtro	0,24 ms	1024 kB

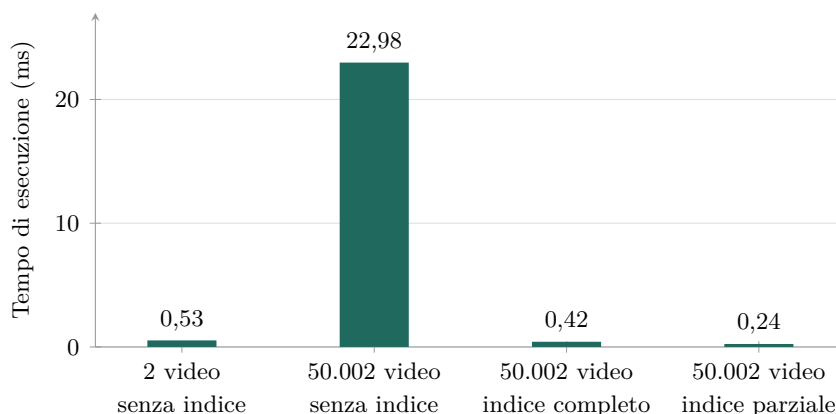


Figura 5.2: Costo dell'interrogazione del feed pubblico nelle quattro configurazioni misurate. L'indice parziale riporta il tempo sotto il quarto di millisecondo.

La prima configurazione descrive la situazione di partenza. Sul catalogo dell'istanza di sviluppo, PostgreSQL sceglie una scansione sequenziale e un ordinamento in memoria, e l'interrogazione dura mezzo millisecondo: su una tabella di poche righe la scansione completa costa meno di un accesso tramite indice. A questo volume, l'assenza dell'indice non è osservabile.

La seconda configurazione mostra cosa succede quando il catalogo cresce. Con cinquantamila video il piano resta lo stesso, ma la scansione sequenziale attraversa l'intera tabella e passa le 46.062 righe pubbliche a un ordinamento parziale di tipo top-N heap-

sort, che ne estrae quarantotto: il tempo passa a 22,98 ms. Il costo cresce in modo lineare con la dimensione del catalogo, mentre il numero di righe restituite resta costante.

La terza configurazione introduce un indice B-tree su `created_at` in ordine decrescente. La scansione cambia da sequenziale a un Index Scan: legge le righe già nell'ordine richiesto e si ferma dopo le prime quarantotto utili, senza alcun ordinamento. Il tempo scende a 0,42 ms. Il piano riporta anche una riga `Rows Removed by Filter: 3`: l'indice contiene tutti i video, compresi quelli privati e quelli bloccati, che vengono letti e poi scartati.

La quarta configurazione sostituisce l'indice con un indice parziale, ristretto alle righe che soddisfano il predicato del feed. Le righe non pubbliche escono del tutto dall'indice: il tempo scende a 0,24 ms e l'indice occupa 1024 kB al posto di 1112 kB.

L'indice della quarta configurazione è stato aggiunto allo schema in `db-init/01_schema.sql`, come riportato nel Codice 5.2, così che ogni installazione della piattaforma lo abbia disponibile fin dal primo avvio.

```
1 CREATE INDEX idx_videos_created_at ON videos (created_at DESC)
2 WHERE is_private = FALSE AND is_flagged = FALSE;
```

Codice 5.2: Indice parziale aggiunto allo schema (`db-init/01_schema.sql`).

5.4.3 Pool di connessioni

Il pool di connessioni descritto nella Sezione 4.1.4 non ha ricevuto alcuna misurazione, poiché il modulo `db/db.js` lo istanzia al bootstrap dell'applicazione e non esiste una configurazione alternativa.

Il load test fornisce alcune informazioni. Durante le prove il server ha sostenuto fra le 393 e le 1928 richieste al secondo per dieci secondi, senza errori di connessione e senza alcun peggioramento progressivo dei tempi di risposta: i percentili restano vicini alla media, e il novantacinquesimo percentile non supera mai il doppio del valore medio. Un comportamento del genere è compatibile con un pool che riutilizza connessioni già aperte. Se ogni richiesta aprisse una nuova connessione a PostgreSQL, il costo del three-way handshake e dell'autenticazione al database porterebbe a un aumento dei tempi di risposta.

5.5 Sintesi delle valutazioni

La valutazione fornisce tre risultati.

Sul piano funzionale, tutti i 58 casi della suite passano e coprono i tre ruoli della piattaforma. La parte dedicata alle politiche di autorizzazione verifica, su endpoint rappresentativi di ciascun ruolo, che le richieste prive dei permessi necessari vengano

rifutate con il codice corretto: questo conferma, sugli endpoint provati, la separazione dei ruoli in NETMED descritta nel Capitolo 4.

I tempi di risposta degli endpoint si collocano tra i 5 e i 26 millisecondi su un catalogo di cinquantamila contenuti, valori compatibili con un uso interattivo. L'endpoint di autenticazione è un ordine di grandezza più lento, e la misura di bcrypt isolato mostra che l'ottantuno per cento di quel tempo è il costo della funzione di hash.

Sul piano delle interrogazioni, il confronto fra le quattro configurazioni del feed pubblico ha individuato un indice mancante e ne ha misurato l'effetto, portando a una modifica dello schema.

I valori riportati sono stati ottenuti senza introdurre livelli di cache né architetture distribuite: derivano dallo schema relazionale, dal pool di connessioni e da un singolo indice.

6

Conclusioni e sviluppi futuri

In questa tesi viene presentato NETMED, una piattaforma web di streaming per contenuti medico-scientifici. Il Capitolo 1 affronta una problematica comune a molte piattaforme: le informazioni sanitarie online non sono sempre affidabili e non esiste alcun modo di verificare chi pubblica i video. L'analisi del Capitolo 2 mostra come le piattaforme già esistenti non colmino questo vuoto. Quelle generaliste sono aperte a tutti, ma verificano solo l'identità dell'autore; quelle specialistiche, invece, verificano la qualifica, ma richiedono un abbonamento e sono rivolte principalmente ai professionisti. NETMED si posiziona fra le due opzioni con l'obiettivo di sfruttare i vantaggi di entrambi i tipi di piattaforma.

6.1 Risultati raggiunti

Il Capitolo 2 individua quattro requisiti necessari alla piattaforma: apertura al pubblico non specializzato, servizio gratuito, verifica della qualifica professionale del creator e libera pubblicazione. La Tabella 2.1 mostra come nessuna delle piattaforme analizzate soddisfi l'insieme di questi requisiti. NETMED offre quindi un sistema gratuito aperto a tutti, in cui, prima di poter pubblicare contenuti nella piattaforma, viene effettuato un controllo che riguarda il titolo dichiarato e non solo la verifica dell'identità. Il creator può pubblicare video in qualsiasi momento senza che ogni contenuto richieda l'approvazione dell'amministratore. Il principale contributo del lavoro consiste nell'integrare in

un'unica piattaforma l'accessibilità al pubblico, la libertà di pubblicazione e la verifica della qualifica professionale del creator.

Il sistema descritto nel Capitolo 4 è eseguito in due container orchestrati da Docker Compose. L'architettura a tre livelli e la separazione dell'area utente da quella amministratore hanno contribuito a tenere in ordine tutte le ventisei pagine HTML, i moduli JavaScript e i quattro moduli di routing. Al primo avvio della piattaforma saranno già presenti lo schema del database e un amministratore, senza che sia necessaria alcuna configurazione iniziale.

Il meccanismo di verifica è il tratto distintivo della piattaforma e si svolge in due fasi: l'utente dichiara titolo, specializzazione e organizzazione di appartenenza al momento della registrazione, in seguito l'amministratore valuta la richiesta e decide se accettarla o meno. La moderazione a strike descritta nella Sezione 4.4.4 affianca la verifica. Quando un commento segnalato viene rimosso, il suo autore riceve uno strike; al terzo l'account passa allo stato **banned** e resta sospeso fino a un intervento dell'amministratore. Le notifiche interne descritte nella Sezione 4.8 avvisano l'utente delle operazioni che lo coinvolgono, senza dipendere da un server di posta.

Nel Capitolo 5 sono riportate le misurazioni delle prestazioni del sistema. I cinquantotto casi della suite automatica passano e coprono tutti i ruoli presenti nella piattaforma. I tempi di risposta stanno fra i cinque e i ventisei millisecondi su un catalogo di cinquantamila video. L'endpoint con le prestazioni minori è quello di login, poiché utilizza la funzione hash bcrypt, che occupa l'ottantuno per cento del suo tempo. Il risultato non richiede alcuna cache; bastano lo schema relazionale, il pool di connessioni e un indice parziale, individuato confrontando i piani di esecuzione.

6.2 Limiti dell'attuale implementazione

6.2.1 Limiti funzionali

I video non sono salvati in NETMED, ma vengono incorporati dal lettore YouTube. La scelta è stata motivata nella Sezione 3.5: il caricamento diretto avrebbe richiesto uno storage a oggetti. Il problema della piattaforma è che non può ospitare contenuti che non siano già su YouTube, e resta esposta alle policy di un servizio di terzi: se un video viene rimosso là, in NETMED resta uno spazio vuoto.

La verifica delle qualifiche è una revisione manuale di tutto ciò che l'utente dichiara. Non è presente alcun tipo di integrazione con gli albi professionali. La barriera in ingresso è quindi più alta rispetto a quella delle spunte blu offerte da altre piattaforme, ma resta comunque un sistema semplice che punta sulla buona fede del richiedente e sul giudizio dell'amministratore. È il limite più importante rispetto alla tesi sostenuta nel Capitolo 1.

Un altro limite già annunciato nell'introduzione riguarda il controllo di chi pubblica, e non di cosa pubblica. Un creator regolarmente iscritto all'albo potrebbe diffondere

contenuti errati, e la piattaforma non avrebbe modo di accorgersene se non tramite le segnalazioni degli utenti.

6.2.2 Limiti tecnici

Il progetto gira in locale su HTTP, senza TLS. In un ambiente reale il token viaggierebbe in chiaro, per questo un certificato e il reindirizzamento su HTTPS sono i primi interventi da effettuare.

Il token è salvato nel `localStorage` del browser. È la soluzione più semplice per un'API stateless, ma lo espone a un attacco XSS, mentre un cookie `HttpOnly` non sarebbe leggibile da JavaScript. Non è inoltre prevista alcuna revoca: un token firmato resta valido per ventiquattro ore, anche se nel frattempo l'utente viene bannato. È il costo della natura stateless discussa nel Capitolo 3, che il middleware `requireCreator` aggira solo in parte, rileggendo i flag dal database a ogni richiesta.

Il rate limiter tiene i contatori in memoria dentro il processo, quindi funziona correttamente con una sola istanza. Con due istanze dietro un bilanciatore di carico, che distribuisce le richieste tra le copie del server, ognuna avrebbe il proprio conteggio e la soglia effettiva raddoppierebbe. Inoltre nessuno dei cinquantotto test verifica il codice 429: il limitatore viene disattivato durante i test proprio perché li bloccherebbe, e il suo comportamento è stato verificato manualmente.

I conteggi delle visualizzazioni, dei like e dei commenti sono calcolati con sottoquery a ogni interrogazione. La scelta mantiene i dati sempre coerenti e alle dimensioni attuali ha un costo contenuto, come mostra il Capitolo 5. Il costo però cresce con il catalogo: a un ordine di grandezza superiore converrebbe salvare quei valori in una colonna dedicata oppure metterli in cache.

L'architettura prevede una sola istanza sia del server sia del database. Il token stateless rende il server applicativo replicabile senza modifiche, ma la scalabilità del livello di persistenza richiederebbe una configurazione master-replica di PostgreSQL.

6.3 Sviluppi futuri

Gli sviluppi elencati sono ordinati dal più semplice al più complesso.

Notifiche esterne. Le notifiche sono consultabili solo dentro la piattaforma: chi non accede regolarmente non viene a sapere nulla di ciò che lo riguarda. Basterebbe estendere con un secondo invio la funzione `createNotification`, descritta nella Sezione 4.8.2, per aggiungere un canale e-mail dedicato agli eventi principali, come l'approvazione della verifica o la sospensione dell'account. Il modulo per la posta esiste già, perché serve alla conferma dell'indirizzo.

Sicurezza per la pubblicazione online. Gli interventi necessari sono tre: HTTPS davanti al server; un access token di breve durata accompagnato da un refresh token in

cookie `HttpOnly`; i contatori del rate limiter spostati su un archivio condiviso come Redis. Nella stessa direzione va la revisione periodica del cost factor di bcrypt, o il passaggio ad Argon2 discusso nella Sezione 3.1.4.

Integrazione con gli albi professionali. È l'evoluzione naturale del contributo principale, e porterebbe il lavoro di tesi su un piano superiore. In Italia la FNOMCeO per i medici e la FNOPI per gli infermieri pubblicano gli elenchi degli iscritti all'albo, consultabili online. Confrontare con questi elenchi il nominativo dichiarato al momento della richiesta ridurrebbe il margine di errore: l'amministratore si limiterebbe a supervisionare il sistema, intervenendo solamente nei casi dubbi. L'unico punto da capire per implementare tale sistema è come effettuare le interrogazioni da un programma.

Moderazione assistita. La moderazione viene effettuata dall'amministratore e dal creator, e non regge alla crescita della piattaforma. Una prima modifica potrebbe essere un filtro su liste di parole che segnali all'amministratore i commenti sospetti. Un secondo livello sarebbe l'utilizzo di un classificatore addestrato su contenuti sanitari per individuare le affermazioni contrarie alle evidenze scientifiche. Questa strada è molto più complicata, poiché un falso positivo silenzia un professionista che ha ragione: il classificatore andrebbe quindi validato prima di affidargli qualsiasi decisione automatica.

Applicazione mobile. L'API REST descritta nella Sezione 4.5 è già utilizzabile da client diversi dal browser, e l'autenticazione a token non andrebbe riprogettata. Un'applicazione nativa per iOS e Android sarebbe un completamento di tutto ciò che è stato implementato.

Architettura distribuita. Con l'aumento degli utenti si passerebbe a più istanze del server dietro un bilanciatore e a PostgreSQL in master-replica, per separare il carico di lettura da quello di scrittura.

6.4 Considerazioni finali

Il lavoro presentato è un prototipo funzionante, non un servizio attivo. Serve a mostrare che il modello è realizzabile e quanto costa costruirlo.

All'inizio del progetto si è dato per scontato che la difficoltà principale fosse di natura tecnica, cioè l'uso di tutti gli strumenti impiegati nella piattaforma, come Node.js, PostgreSQL e Docker. Sono però strumenti maturi e ben documentati, e i problemi che pongono hanno soluzioni note. La difficoltà vera, invece, è stata aggirare l'indisponibilità di materiale video di ambito clinico, dovuta ai vincoli di tutela dei dati personali. Da qui la scelta, descritta nella Sezione 3.5, di riprodurre i contenuti attraverso il player di YouTube.

Questo vincolo ha portato a un risultato che si ritiene il più importante del percorso. Non potendo intervenire sui contenuti, la garanzia offerta all'utente poteva poggiare soltanto su chi li pubblica. È la stessa distinzione fatta nel Capitolo 2, parlando delle

piattaforme generaliste, dove non si critica il servizio di distribuzione, ma l'assenza di un controllo sulla qualifica di chi carica i video. NETMED interviene sulla qualifica di chi pubblica e lascia la distribuzione a un servizio esterno.

Il contributo del lavoro non dipende dalla scelta di appoggiarsi a YouTube. Se in futuro la piattaforma ospitasse i video direttamente, il meccanismo di verifica dei creator rimarrebbe invariato; cambierebbe solo l'approccio di distribuzione dei contenuti, e non il criterio con cui si stabilisce chi può pubblicarli.

NETMED non risolve il problema della disinformazione sanitaria. Mostra però che una piattaforma di divulgazione sanitaria può essere costruita in modo diverso da quelle che esistono e si utilizzano oggi, mantenendo la verifica della fonte tra i requisiti di partenza invece che tra le funzioni secondarie.

Bibliografia

- [1] Eurostat, “EU citizens: Over half seek health information online,” 2022. Dati dell’indagine 2021, dataset isoc_ci_ac.i. <https://ec.europa.eu/eurostat/web/products-eurostat-news/-/edn-20220406-1>.
- [2] D. J. Welbourne and W. J. Grant, “Science communication on YouTube: Factors that affect channel and video popularity,” *Public Understanding of Science*, vol. 25, no. 6, pp. 706–718, 2016. <https://doi.org/10.1177/0963662515572068>.
- [3] K. C. Madathil, A. J. Rivera-Rodriguez, J. S. Greenstein, and A. K. Gramopadhye, “Healthcare information on YouTube: A systematic review,” *Health Informatics Journal*, vol. 21, no. 3, pp. 173–194, 2015. <https://doi.org/10.1177/1460458213512220>.
- [4] E. K. Vraga and L. Bode, “Defining misinformation and understanding its bounded nature: Using expertise and evidence for describing misinformation,” *Political Communication*, vol. 37, no. 1, pp. 136–144, 2020. <https://doi.org/10.1080/10584609.2020.1716500>.
- [5] D. J. Rothkopf, “When the buzz bites back,” 2003. The Washington Post, 11 maggio 2003. Prima attestazione del termine “infodemic”. <https://www.washingtonpost.com/archive/opinions/2003/05/11/when-the-buzz-bites-back/bc8cd84f-cab6-4648-bf58-0277261af6cd/>.
- [6] World Health Organization, “Managing the COVID-19 infodemic: Promoting healthy behaviours and mitigating the harm from misinformation and disinformation,” 2020. <https://www.who.int/news/item/23-09-2020-managing-the-covid-19-infodemic-promoting-healthy-behaviours-and-mitigating-the-harm-from-misinformation-and-disinformation>.
- [7] OpenJS Foundation, “Express — fast, unopinionated, minimalist web framework for Node.js,” 2025. <https://expressjs.com/>.
- [8] Statista, “Leading websites worldwide, by monthly visits,” 2025. <https://www.statista.com/statistics/1201880/most-visited-websites-worldwide/>.

-
- [9] YouTube, “YouTube press — by the numbers,” 2025. <https://blog.youtube/press/>.
 - [10] H. O.-Y. Li, A. Bailey, D. Huynh, and J. Chan, “YouTube as a source of information on COVID-19: A pandemic of misinformation?,” *BMJ Global Health*, vol. 5, no. 5, p. e002604, 2020. <https://pubmed.ncbi.nlm.nih.gov/32409327/>.
 - [11] Google LLC, “YouTube Health: Bringing more medical information to YouTube,” 2021. <https://health.youtube/>.
 - [12] M. Zenone, N. Ow, and S. Barbic, “TikTok and public health: A proposed research agenda,” *BMJ Global Health*, vol. 6, no. 11, p. e007648, 2021. <https://pubmed.ncbi.nlm.nih.gov/34819326/>.
 - [13] Fisioterapia Interattiva S.r.l., “Streamed — piattaforma di formazione per terapisti,” 2025. <https://streamededu.com/>.
 - [14] Agenas — Agenzia Nazionale per i Servizi Sanitari Regionali, “Educazione Continua in Medicina: Normativa,” 2025. <https://ape.agenas.it/ecm/normativa.aspx>.
 - [15] Elsevier B.V., “Osmosis — medical education for students,” 2025. <https://www.osmosis.org/>.
 - [16] WebMD Health Corp., “Medscape — medical news and continuing medical education,” 2025. <https://www.medscape.com/>.
 - [17] Coursera, Inc., “Coursera — online courses from top universities,” 2025. <https://www.coursera.org/>.
 - [18] edX Inc., “edX — free online courses by Harvard, MIT and more,” 2025. <https://www.edx.org/>.
 - [19] MEDtube, “MEDtube — medical videos sharing platform for healthcare professionals,” 2025. <https://medtube.net/>.
 - [20] M. B. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Tech. Rep. RFC 7519, IETF, 2015. <https://www.rfc-editor.org/rfc/rfc7519>.
 - [21] M. B. Jones, “JSON Web Algorithms (JWA),” Tech. Rep. RFC 7518, IETF, 2015. Definisce l’algoritmo HS256. <https://www.rfc-editor.org/rfc/rfc7518>.
 - [22] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” Tech. Rep. RFC 2104, IETF, 1997. <https://www.rfc-editor.org/rfc/rfc2104>.

- [23] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” Tech. Rep. FIPS PUB 180-4, U.S. Department of Commerce, 2015. <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.180-4.pdf>.
- [24] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proceedings of the 1999 USENIX Annual Technical Conference*, pp. 81–91, 1999. <https://www.usenix.org/conference/1999-usenix-annual-technical-conference/future-adaptable-password-scheme>.
- [25] A. Biryukov, D. Dinu, and D. Khovratovich, “Argon2: New generation of memory-hard functions for password hashing and other applications,” in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 292–302, IEEE, 2016. <https://doi.org/10.1109/EuroSP.2016.31>.
- [26] OWASP Foundation, “Password storage cheat sheet,” 2025. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.
- [27] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970. <https://doi.org/10.1145/362384.362685>.
- [28] T. Haerder and A. Reuter, “Principles of transaction-oriented database recovery,” *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, 1983. <https://doi.org/10.1145/289.291>.
- [29] D. Pritchett, “BASE: An ACID alternative,” *ACM Queue*, vol. 6, no. 3, pp. 48–55, 2008. <https://doi.org/10.1145/1394127.1394128>.
- [30] The PostgreSQL Global Development Group, “PostgreSQL: The world’s most advanced open source relational database,” 2025. <https://www.postgresql.org/>.
- [31] The PostgreSQL Global Development Group, “PostgreSQL 16 documentation: JSON types,” 2025. <https://www.postgresql.org/docs/16/datatype-json.html>.
- [32] HTTP Archive, “The Web Almanac: JavaScript,” 2024. <https://almanac.httparchive.org/en/2024/javascript>.
- [33] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, p. 2, 2014. <https://dl.acm.org/doi/10.5555/2600239.2600241>.

- [34] S. Tilkov and S. Vinoski, “Node.js: Using JavaScript to build high-performance network programs,” *IEEE Internet Computing*, vol. 14, no. 6, pp. 80–83, 2010. <https://doi.org/10.1109/MIC.2010.145>.
- [35] Docker, Inc., “Docker Compose overview,” 2025. <https://docs.docker.com/compose/>.
- [36] Carlson, Brian and contributors, “node-postgres — PostgreSQL client for Node.js,” 2025. <https://node-postgres.com/>.
- [37] OWASP Foundation, “Authentication cheat sheet,” 2025. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html.
- [38] OWASP Foundation, “Session management cheat sheet,” 2025. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html.
- [39] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. <https://roy.gbiv.com/pubs/dissertation/top.htm>.
- [40] R. T. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” Tech. Rep. RFC 9110, IETF, 2022. <https://www.rfc-editor.org/rfc/rfc9110>.
- [41] OpenJS Foundation, “Mocha — the fun, simple, flexible JavaScript test framework,” 2025. <https://mochajs.org/>.
- [42] Chai.js contributors, “Chai — BDD / TDD assertion library for Node.js,” 2025. <https://www.chaijs.com/>.
- [43] Ladjs contributors, “SuperTest — HTTP assertions made easy via SuperAgent,” 2025. <https://github.com/ladjs/supertest>.
- [44] Collina, Matteo and contributors, “Autocannon — fast HTTP/1.1 benchmarking tool written in Node.js,” 2025. <https://github.com/mcollina/autocannon>.

Ringraziamenti

Ringrazio la prof.ssa Alessandra Rizzardi, che mi ha accompagnato nella realizzazione di questo elaborato, per la disponibilità, la pazienza e il supporto dimostrati durante tutto il percorso.

Un grazie speciale va ai miei amici e ai miei colleghi, che in questi anni mi sono stati accanto e mi hanno supportato. Ho incontrato ognuno di voi in ambienti diversi e mi siete rimasti impressi, ciascuno a modo suo: siete persone fantastiche, che mi avete fatto passare momenti stupendi tra risate e impegni. Grazie per aver reso questi anni più leggeri, più divertenti e pieni di ricordi che porterò sempre con me.

Dal più profondo del mio cuore ringrazio i miei genitori, che mi avete dato la possibilità di studiare, per tutti i sacrifici che avete fatto per me e per aver creduto in me. Anche nei momenti in cui sono caduto mi avete aiutato a rialzarmi. Siete e sarete fondamentali per il mio percorso e per la mia crescita.

Non posso non ringraziare i miei fratelli Mohit e Rohit, mia cognata Manpreet e tutta la mia famiglia, compresi i miei cagnoloni, che mi avete sostenuto e ispirato ogni giorno durante questo viaggio. Sarete sempre il mio punto di riferimento.

Infine vorrei ringraziare una persona davvero speciale, Carlotta, per avermi supportato nei momenti di crisi e disperazione, per avermi aiutato lungo la strada, per la spensieratezza, le risate e la costante vicinanza anche quando le cose si facevano difficili. Sei e sarai fondamentale nella mia vita.

unha lay jinha nu koshish karan da mauka ni mileya

a chi non ha avuto la possibilità di provarci